

筑波大学大学院博士課程

理工情報生命学術院

システム情報工学研究群修士論文

Maintenance Policies for Mitigating Imperfect Retraining in Entangled Machine Learning Systems

Zhengji Wang

修士（工学）

（情報理工学位プログラム）

指導教員 町田文雄（Fumio Machida）

2025 年 3 月

Abstract

Machine Learning Systems (MLSs) consist of interconnected models working collaboratively to achieve complex objectives. Over time, these systems are prone to performance degradation due to dataset shifts, necessitating periodic retraining of individual component models. However, retraining a component model can negatively influence the performance of dependent ML models in the system. This issue is rooted in the entanglement of ML model components. Although the issue has been recognized in the literature, the existing studies have two limitations: 1. Only a few works addressed the issue of imperfect retraining with real ML application examples, 2. the existing studies did not provide any solutions to the imperfect retraining problem. This thesis addresses these limitations by providing case studies, comprehensive theoretical analysis of imperfect retraining, and strategies to mitigate its impact.

To address the lack of real-world examples, we introduced two types of imperfect retraining that might affect the MLS negatively. We further conducted empirical studies using tasks from computer vision and natural language processing. These examples highlight the challenges posed by model entanglement. Specifically, we observed that improvements in the performance of an upstream model can, in certain scenarios, lead to a deterioration in overall system performance. This counterintuitive outcome underscores the severity and practical significance of the problem, demonstrating how tightly coupled components in MLSs can negatively interact during retraining.

To provide practical guidance on mitigating the imperfect retraining problem, we analyze two baseline retraining strategies: a progressive policy, which emphasizes frequent updates, and a conservative policy, which prioritizes system stability. By modeling these strategies using continuous-time Markov chains, we identify conditions under which each policy enhances service availability.

To broaden the scope of our analysis, we extend the model to a semi-Markov process, which offers greater flexibility in capturing the complexities of transition dynamics. In addition, we define service continuity to measure the stability of system performance over time, complementing traditional availability metrics. Our numerical analysis shows that the progressive retraining policy generally achieves higher service availability by adapting quickly to dataset shifts, whereas the conservative retraining policy ensures greater service continuity by minimizing performance fluctuations. This trade-off highlights the need to carefully balance the two objectives when selecting a retraining strategy.

Building on these insights, we propose a novel dynamic retraining policy to address the trade-off between service availability and continuity. This policy dynamically decides whether to proceed with or defer retraining attempts based on the current state of the system. Numerical results validate its effectiveness, demonstrating that the dynamic policy outperforms both progressive and conservative retraining strategies by maximizing overall system utility. This study not only enhances the theoretical understanding of retraining in MLSs but also provides practical strategies to improve their reliability and adaptability, paving the way for more robust MLSs.

Contents

1	Introduction	1
2	Related Work	4
2.1	Dataset Shift and Its Challenges	4
2.2	Operations and Maintenance in MLSs	4
2.3	Machine Learning Component Entanglement	5
2.4	Imperfect Retraining in MLSs	5
2.5	Availability Analysis in System Modeling	5
3	Problem Setting	7
3.1	Machine Learning System	7
3.2	Imperfect Retraining	8
4	Case Studies of Imperfect Retraining	10
4.1	Pseudo-Lidar System for 3D Object Detection in Autonomous Driving	10
4.2	Sentence Classification System for Social Media	12
5	Progressive and Conservative Retraining Policy	15
5.1	Motivation	15
5.2	State Space Modeling	15
5.2.1	Definition	15
5.2.2	Progressive Retraining Policy	16
5.2.3	Conservative Retraining Policy	17
5.3	Numerical Analysis	18
5.3.1	Research Question	18
5.3.2	Parameter Assignment	18
5.3.3	Sensitivity Analysis	19
5.4	Findings	20
6	Trade-off between Progressive and Conservative Retraining Policy	21
6.1	Motivation	21
6.2	State Space Modeling	21
6.2.1	Definition	21
6.2.2	Progressive Retraining Policy	22

6.2.3	Conservative Retraining Policy	24
6.2.4	Metrics Definition	25
6.3	Numerical Analysis	26
6.3.1	Research Questions	26
6.3.2	Parameter Assignment using Weibull Holding Time	26
6.3.3	RQ1: Sensitivity Analysis	28
6.3.4	RQ2: Comparison to CTMC	29
6.3.5	RQ3: Trade-off analysis: Availability vs Continuity	30
6.4	Strategy Analysis	31
7	Dynamic Retraining Policy	33
7.1	Motivation	33
7.2	Mechanism of Dynamic Retraining	34
7.3	State Space Modeling	36
7.3.1	Definition	36
7.3.2	Dynamic Retraining Policy	36
7.4	Performance Analysis	38
7.5	Utility Analysis	40
7.5.1	Utility Function	40
7.5.2	Maximum Utility Analysis	41
7.6	Strategy Analysis	43
8	Conclusion	45
	Acknowledgment	47
	References	48

List of Figures

3.1	An MLS with two components	7
3.2	Type 1 and Type 2 component entangled enhancement in the problem setting . . .	9
4.1	MLS for computer vision	10
5.1	CTMC for progressive retraining policy	16
5.2	CTMC for dynamic retraining policy	17
5.3	Service Availability regarding to different parameter changes	20
6.1	SMP for progressive retraining policy	22
6.2	SMP for conservative retraining policy	24
6.3	Service Availability regarding to different shape parameters	29
6.4	Service Availability and Service Continuity regarding to different parameter changes	30
6.5	A strategy to select the retraining policy for the MLS	32
7.1	Mechanism for dynamic retraining policy	34
7.2	SMP for dynamic retraining policy	36
7.3	Different ROCs	38
7.4	Service Availability and Service Continuity regarding to different ROCs	39
7.5	Service Availability and Service Continuity when utility function is convex	41
7.6	Service Availability and Service Continuity when utility function is concave and availability is more important	42
7.7	Service Availability and Service Continuity when utility function is concave and continuity is more important	42
7.8	The ROC of the evaluator	44
7.9	Service Availability and Service Continuity when evaluation method has moderate performance	44

Chapter 1

Introduction

Machine Learning (ML) has progressed dramatically over the past two decades, from laboratory curiosity to a practical technology in widespread commercial use [1]. With the fast development of both ML algorithms and computing hardware, ML models have been applied in many application domains such as medical and commercial ads, performing tasks such as classification and clustering [2]. Machine learning system (MLS) is a system that incorporates ML models as its components. MLS allows large teams to effectively divide-and-conquer complex engineering tasks by making each team focusing on a specific component. Examples of MLS includes an image captioning system, a search advertising system, and a personalized recommendation system [3] [4] [5].

Despite their widespread adoption, MLSs face significant challenges in real-world operations, particularly due to dataset shift—a phenomenon where the data distribution changes over time, violating the assumption that training and testing data follow the same distribution [6]. Dataset shift can degrade the performance of individual ML components, necessitating periodic retraining to restore their effectiveness [7]. However, retraining is not always effective and can even harm overall system performance. We define this phenomenon as imperfect retraining [8]. Imperfect retraining can arise due to several factors, including component entanglement, biased selection of training or validation data, or overfitting caused by suboptimal training strategies [9] [10].

In this work, we specifically focuses on the impact of entangled enhancement, a specific form of imperfect retraining caused by the interconnected nature of MLS components [8]. In many practical scenarios, retraining all MLS components jointly is not infeasible or undesirable. For instance, some models in an MLS may be externally developed and fixed, or retraining the entire system may be too time-consuming [9] [11]. As a result, updating a single component may not lead to an improvement in overall system performance, and, in some cases, may even degrade it. This poses a threat to the operations of the MLS. Understanding the way to mitigate the impact of imperfect retraining becomes crucial to maintain the performance of MLS. In spite of this, there are very few studies mentioning such risk and no systematic solution has ever been proposed [8] [12].

To address these limitations, this thesis first give a theoretical problem setting, which formulate the problem of imperfect retraining, by introducing a modeling of two-component sequentially dependent MLS. This framework identifies key factors such as dataset shifts and component updates, defining precise states for each model to enable detailed analysis of their interactions and impact on system performance.

To illustrate the practical challenges of model entanglement, we conducted empirical studies in computer vision and natural language processing. The result showcases how retraining individual components can fail to improve, or even degrade, the performance of MLSs.

Next, we introduce two maintenance policies for retraining MLS components, namely *progressive retraining policy* and *conservative retraining policy*, to help maintain the MLSs against the imperfect retraining. The progressive retraining policy retrains ML models whenever the performance of a component in MLS fails to achieve the required standard, while the conservative retraining policy only retrains the ML models when the overall performance of the system is deteriorated. To compare the effectiveness of the retraining policies, we develop Continuous-time Markov Chains (CTMCs) that can capture the state transitions of an MLS consists of two sequentially-dependent ML models. Through the comprehensive sensitivity analysis on the proposed models, we find that the conservative retraining policy and the progressive retraining policy have advantages in terms of service availability in different conditions.

Although we have found out the impact of retraining policies on service availability, it is still unknown how the distribution of the state transition could affect the system under different retraining policies, as CTMCs limit the state transition solely to be exponentially distributed. To better model the retraining dynamics, we leverage Semi-Markov Processes (SMPs), which allow for greater flexibility in representing transition dynamics compared to CTMCs. SMPs accommodate non-exponential transition times, enabling a more accurate representation of real-world retraining processes. Besides, we define service continuity as a new metric to quantify the expected time that the system can maintain required service quality without interruptions. This metric complements traditional availability metrics, as availability only reflects the long-term average accuracy of the prediction result instead of the stability of the service operation. Through the extensive numerical studies, we found a notable trade-off between service availability and continuity. The conservative retraining policy consistently excels in maintaining service continuity. Meanwhile, the progressive retraining policy generally provides higher service availability across most parameter ranges.

Given the observed trade-off relationship between the two retraining policies, we proposed the *dynamic retraining policy* that selectively use the two baseline retraining policies based on the quality of the retraining attempts. Dynamic retraining policy enables the manager of the MLS to tailor the service availability and continuity by changing the retraining frequency using monitoring mechanism, therefore adapts to different conditions and balances the competing objectives of availability and continuity. To showcase the advantage of the dynamic retraining, we introduced utility function to model the objective for the performance the MLS, and the numerical analysis reveals that the dynamic retraining has advantage in achieving higher utility compared to progressive and conservative retraining policies.

In summary, this thesis makes the following contributions:

- We define the problem of imperfect retraining of entangled MLSs in two types.
- We showcase the real problem of entangled enhancement through the experiments of 3D object detection system and sentence classification system.
- We propose two maintenance policies, the progressive and the conservative retraining policies, and evaluate the service availability through the analysis of CTMCs representing the state transitions of the MLS.

- We introduce the service continuity as a metric and SMPs as modeling method to reveal the notable trade-off between service availability and service continuity.
- We propose the dynamic retraining policy based on the two baseline policies to maximize the utility for the MLS.

The rest of the thesis is organized as follows. Chapter 2 presents related work. Chapter 3 explains the problem setting in detail. Chapter 4 showcases the examples of imperfect retraining in different scenarios. Chapter 5 proposes the availability modeling and numerical analysis of the MLS using CTMC for both progressive retraining policy and conservative retraining policy. Chapter 6 proposes the state-space models of the MLS maintenance using SMP and the numerical analysis of the MLS under retraining policies. Chapter 7 proposes the dynamic retraining policy and its utility maximum property in maintaining the MLSSs. Finally, in Chapter 8, we provide our conclusion.

Chapter 2

Related Work

The related work can be categorized into the following groups of studies: dataset shift and its challenges, operations and maintenance in MLSs, machine learning component entanglement, imperfect retraining in MLSs, availability analysis in system modeling.

2.1 Dataset Shift and Its Challenges

Dataset shift, also known as concept drift, occurs when the distribution of training data differs from that of testing data, violating the fundamental assumption that both datasets follow the same underlying distribution [6]. This issue has been widely studied in the ML community and is recognized as a critical challenge to maintaining ML model performance over time [13] [14]. Common causes of dataset shift include sample selection bias, where the training data is non-uniformly sampled, and non-stationary environments, where data distributions evolve due to temporal or spatial changes [6]. These shifts can lead to performance degradation, making periodic interventions, such as retraining, essential to ensure long-term operational effectiveness.

To counteract dataset shift, various approaches have been proposed, including unsupervised techniques to monitor for changes in data and retraining models when shifts are detected [15] [16]. However, these methods typically focus on individual ML models without accounting for the broader context of MLSs, which often consist of multiple interdependent components. In MLSs, retraining a single model to address dataset shift may inadvertently disrupt the performance of other components, introducing new risks that are often overlooked in traditional single-model scenarios [11]. This gap highlights the need for a system-wide perspective on mitigating dataset shift in MLSs.

2.2 Operations and Maintenance in MLSs

Despite the rapid advancement of ML in recent years, many ML proofs of concept have not progressed to production due to insufficient emphasis on operational techniques [17]. Manual operation of ML models is challenging due to the complexity of software and hardware components, necessitating robust automation [18]. Additionally, models must be retrained on new data due to dataset shifts, further emphasizing the need for automation [19]. Machine Learning Operations (MLOps)

is a developing paradigm in recent years in an effort to address these needs by ensuring reliable deployment and maintenance of models [20]. However, automation in MLSs faces challenges due to component entanglement.

2.3 Machine Learning Component Entanglement

Component entanglement is a unique challenge in MLSs, where the outputs of upstream models serve as inputs for downstream models. This interconnectedness creates complex dependencies, making it difficult to attribute system-level failures to individual components [9]. A particularly problematic manifestation of entanglement is self-defeating improvement, where enhancements to an upstream model may fail to improve—or even degrade—the performance of downstream components [12]. This phenomenon contrasts sharply with traditional software systems, where improvements to individual components typically result in better overall system performance [21].

In many MLSs, the modular nature of components often necessitates independent training and retraining due to constraints such as proprietary ownership, computational costs, or operational complexity. Consequently, joint optimization of all components is frequently infeasible, further exacerbating the risks posed by entanglement during retraining [12] [11]. These unique characteristics of MLSs underscore the need for tailored strategies to mitigate the challenges of component entanglement.

2.4 Imperfect Retraining in MLSs

Imperfect retraining is a phenomenon where the retraining of one or more MLS components fails to improve the overall system performance, often due to issues such as component entanglement, biased training data selection, or overfitting [10]. Existing efforts to address imperfect retraining typically focus on algorithmic improvements, monitoring mechanisms, or post-retraining adjustments to the models themselves [12] [6] [9]. While these approaches aim to enhance the reliability of ML models, they often overlook strategies for mitigating the system-wide impact of imperfect retraining.

In contrast, our study adopts a system-level perspective, emphasizing the need to minimize the adverse effects of imperfect retraining on MLSs. This approach is inspired by reliability engineering practices in software systems but is adapted to account for the unique characteristics of MLSs, where component functionality is less explicit and performance improvements are not guaranteed to propagate throughout the system [21] [22].

2.5 Availability Analysis in System Modeling

Availability, a key metric for system dependability, measures the ability of a system to deliver services as required. To analyze service availability, stochastic models such as Continuous-Time Markov Chains (CTMCs) are commonly used [23] [24]. These models have been applied in various domains, including telecommunications, computer systems, and software reliability. However, the

assumptions underlying CTMCs, particularly the use of exponential distributions for state sojourn times, may not accurately capture real-world dynamics.

To address these limitations, Semi-Markov Processes (SMPs) have been introduced as a more flexible modeling approach. By allowing arbitrary state sojourn time distributions, SMPs have been successfully applied to problems such as software aging, manufacturing reliability, and power supply availability [25] [26] [27].

To the best of our knowledge, our work represents the first application of CTMCs and SMPs to model an MLS with retraining policies. Additionally, while prior studies have focused primarily on service availability, our work introduces the metric of service continuity, which captures the stability of MLS performance over time by quantifying interruptions during retraining. This dual focus on availability and continuity provides a more comprehensive framework for evaluating MLS reliability.

Chapter 3

Problem Setting

In this section, we will define the MLS under examined and the problem of imperfect retraining.

3.1 Machine Learning System

We consider a Machine Learning System (MLS) consisting of multiple ML models structured as a static directed acyclic graph (DAG), $G = (\mathcal{V}, \mathcal{E})$. Each vertex $v \in \mathcal{V}$ represents an ML model, and each edge $(v, w) \in \mathcal{E}$ represents the output of model v being used by model w . Given an edge (v, w) , v is referred to as the *upstream model*, and w as the *downstream model*.

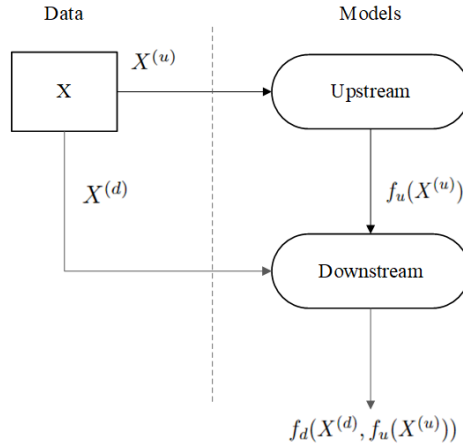


Figure 3.1: An MLS with two components $\mathcal{V} = \{u, d\}$, where u is upstream of d , and the output for the system becomes $f_d(f_u(X^{(u)}), X^{(d)})$.

Fig. 3.1 illustrates an MLS with two components: an upstream model u and a downstream model d . Each vertex in G applies a function on its input data and the output from its upstream models. The function for vertex v is denoted as $f_v(x)$. For the parent node u , which has no upstream model, the input is derived from a subset of the original data X , denoted as $X^{(u)}$. The upstream model processes $X^{(u)}$ to produce $f_u(X^{(u)})$, which, along with another subset of the input $X^{(d)}$, becomes the

input for the downstream model d . The system’s output can be represented as $f_d(f_u(X^{(u)}), X^{(d)})$.

An example of such a system was presented in [12], where an upstream depth estimation model infers the z-coordinate from 2D images, and a downstream object detection model uses the 3D information to detect the location of cars in 3D coordinates. Interestingly, after retraining the upstream model to decrease mean error in the z-coordinate, the downstream model experienced a performance deterioration in 3D detection accuracy.

Performance The performance of an MLS is periodically evaluated using an ideal hypothesis dataset, denoted as X_{real} , which faithfully represents the latest real-world data. This dataset contains data points $\{(x'_1, y'_1), \dots, (x'_{n_2}, y'_{n_2})\}$. The performance metric could be Precision, Recall, or Mean Average Error, depending on the task. Each ML model must satisfy a predefined performance threshold for the target metric. While dataset shift may cause performance degradation, the threshold is assumed to be achievable through multiple retraining iterations.

Dataset Shift Initially, both upstream model u and downstream model d satisfy their respective performance thresholds, making the system capable of handling real-world tasks. Over time, the real-world distribution represented by X_{real} changes due to dataset shift, leading to performance degradation in both u and d . If either model’s performance drops below its threshold, it is considered a failure for that specific component.

Training For training, a dataset X_{train} , consisting of data points $\{(x_1, y_1), \dots, (x_{n_1}, y_{n_1})\}$, is used. Each model v is trained using a learning algorithm f_v chosen from the hypothesis set H_v . The algorithm, training data, and hypothesis set remain fixed during the training process. Training is conducted separately for each model, meaning that during the training of d , the results are not back-propagated to u .

3.2 Imperfect Retraining

To counter the dataset shift in the process of maintaining the MLS, dataset shifts are monitored, and retraining is triggered when a shift is detected. However, retraining can be imperfect and may fail to benefit the system. Imperfect retraining can be caused by solely the uncertainty of a training procedure. **Degenerative retraining** is defined as the retraining that worsens the component’s performance, failing to benefit the system output. Causes include biased training data, insufficient data, or overfitting. Yet another complicated scenario of imperfect retraining is **entangled enhancement** that can be caused by the dependency of component models. It is defined as the retraining improves the component’s performance but fails to improve the system output due to component entanglement.

In our formulated system which has two ML models as components, we mainly consider the entangled enhancement as the biggest threat. Entangled enhancement is triggered particularly when the upstream model u does not reach the threshold due to dataset shift. For a two-component MLS, this phenomenon manifests as:

1. **Type 1:** The upstream model u fails to meet its threshold, while the downstream model d meets its threshold. Retraining u can degrade d 's performance.
2. **Type 2:** Both u and d fail to meet their thresholds. Retraining u leads to an entangled enhancement where d still cannot meet its threshold.

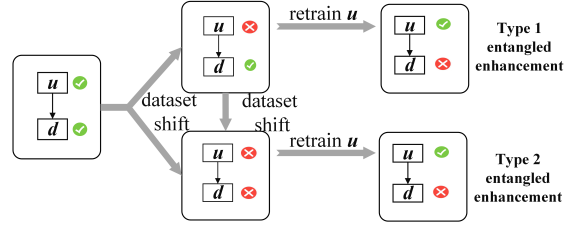


Figure 3.2: Type 1 and Type 2 component entangled enhancement in the problem setting.

In these cases, retraining u may directly improve its performance but negatively impacts the downstream model d . Experiments in Chapter 4 showcase this effect, highlighting how retraining u can ultimately reduce overall system performance. Addressing these challenges requires careful consideration of the interplay between MLS components during retraining.

Chapter 4

Case Studies of Imperfect Retraining

To demonstrate real examples of imperfect retraining, we conduct two experiments of entangled enhancement in the tasks of computer vision and natural language processing respectively.

4.1 Pseudo-Lidar System for 3D Object Detection in Autonomous Driving

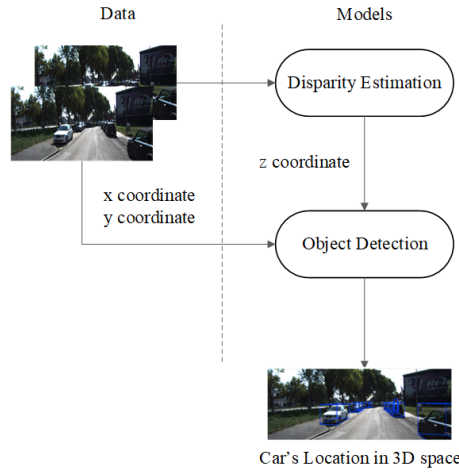


Figure 4.1: The MLS for the experiment regarding to computer vision. The data consists of two images taken by left camera and right camera. The upstream model PSMNet uses the images to predict the depth information. The downstream model Point RCNN uses the depth information provided by the upstream model, together with the x and y information from 2D images, to predict the car's location in the 3D space.

In this experiment, we recreated the phenomenon of imperfect retraining in the form of component entanglement in a 3D object detection system for autonomous driving scenario also considered in [12]. Fig. 4.1 shows a widely applied MLS in the self-driving car setting. The data input are two

images shot from left and right cameras. The system uses a pseudo-LiDAR [28] for the basis of 3D detection of cars. For the upstream model u , PSMNet is adopted [29] to make prediction of the disparity from these two models. The prediction result, together with the original 2D images, would be processed to generate 3D point clouds. For the downstream model, Point R CNN is adopted [30]. The Point R CNN model takes the 3D point clouds as the input, and generates the car’s location in 3D spaces as the output. The dataset for the experiment comes from Kitti Dataset [31]. While the original study in [12] used AP_{BBOX} and AP_{3D} as their chosen metrics, and also conducted experiments on pedestrians, our experiments differ from the previous study as we limit our scenario to a two-model system, thus only paying attention to car’s detection result. Also, we selected AP_{R40} , which use 40 recall positions instead of the 11 recall positions proposed in the original Pascal VOC benchmark [32], thus making this metric a more fair comparison of the results [33]. We also includes bird’s eye view (BEV) and average orientation similarity (AOS), thus incorporating all possible metrics according to the Kitti Dataset standard to make the result more inclusive.

Upstream Model We trained PSMNet with two training loss functions: (1) the depth mean absolute error and (2) the disparity mean absolute error (MAE). Depth is proportional to the reciprocal of the disparity. We assume that the PSMNet trained using depth MAE is the original upstream model, and is denoted as u , and the PSMNet trained using disparity MAE is the retrained model, and is denoted as u' . After training these two versions of models using different loss functions, the MAE of each model for disparity on the validation set is shown in TABLE 4.1.

Table 4.1: The disparity’s MAE before and after the update

PSMNet		
range	original model u	retrained model u'
0-10m	1.61	1.38
10-20m	1.0	1.01
over 20m	1.42	1.50
Whole range	1.28	1.21

It can be observed that after the retraining, indeed the upstream model’s prediction ability increases in the whole range, as the MAE of the disparity decreases from 1.28 to 1.21. The improvement effect is dramatic for close-range objects within 10 meters, in which the MAE of the disparity is reduced from 1.61 to 1.38. However, for objects that are farther from the camera, like from the range of 10 meters to 20 meters, the disparity estimation result actually has a slightly higher MAE, which increases from 1.0 to 1.01. Specifically for objects over 20 meters, the MAE increases from 1.42 to 1.50, showing a drop in the estimation accuracy.

Downstream Model By using the estimation from u and u' , we process two versions of the pseudo-LiDAR point clouds as the inputs, to train two versions of the downstream model, which are both Point R CNN. Besides the input data, there is no change in training algorithm. The result is shown in TABLE 4.2:

The evaluation metric employed for the downstream model is AP_{R40} with an Intersection over

Table 4.2: AP_R40 of detection result at IoU threshold of 0.7

Point R CNN		
metrics	original model u	retrained model u'
3D	44.75	42.62
BEV	56.02	54.06
AOS	76.44	74.15
BBOX	79.17	76.36

Union (IoU) threshold of 0.7. The common practice of setting the IoU threshold to 0.7 aligns with the Kitti Benchmark and is consistent with prevailing standards in diverse object detection literature [30] [28]. In contrast to the traditional AP , which utilizes 11 recall positions as per the original Pascal VOC benchmark, AP_R40 encompasses 40 recall positions, rendering it a more comprehensive measure for assessing object detection outcomes [33]. Four distinct metrics, namely AP_R40_{3D} , AP_R40_{BEV} , AP_R40_{AOS} , and AP_R40_{BBOX} , are employed to evaluate various aspects of detection performance, representing 3D bounding box accuracy, 2D bird’s-eye view accuracy, average orientation similarity, and 2D bounding box accuracy, respectively.

Notably, an assessment of the downstream model based on both AP_R40_{3D} and AP_R40_{BEV} at an IoU threshold of 0.7 reveals that, despite an enhancement in the upstream model’s performance, the resultant output leads to diminished accuracy in the downstream model’s ability to detect the location of cars. Specifically, the downstream model’s performance experiences a decline from 44.75 to 42.62 for AP_R40_{3D} and from 56.02 to 54.06 for AP_R40_{BEV} . Similar deterioration is observed for AP_R40_{AOS} and AP_R40_{BBOX} , which decrease from 76.44 to 74.15 and from 79.17 to 76.36, respectively.

This decline can be attributed to the upstream model’s heightened proficiency in estimating disparity for short-range objects, accompanied with a compromise in its capacity to accurately estimate disparities for objects at a greater distance. Given that cars are typically positioned farther from the camera, they fail to benefit proportionally from the improvement, resulting in a paradoxical performance degradation.

Result Analysis The experiment replicates the complete process of updating an upstream model. The result confirms the phenomenon of the component entanglement indeed can happen in a two-component MLS when upstream model is retrained. In the proposed system, we retrained the upstream model of disparity estimation and reduced its disparity MAE on validation set. However, the downstream model for detecting car actually shows a decrease in precision, thus showcasing the validity of imperfect retraining, which is also the entangled enhancement in a two-component MLS.

4.2 Sentence Classification System for Social Media

In this experiment, we created the phenomenon of the component entanglement in a sentence classification system for information collected from a social media. For the upstream model u , a feature extraction model is adopted to extract the numerical vector from the sentence. We use BERT

and TextCNN as the different versions of the upstream model before and after the retraining [34] [35]. The extracted features, concatenated with the raw features extracted from the sentence using tf-idf as well as other statistical feature such as the number of norms in the sentence, would be used as the input for the downstream model. For the downstream model d , XGBoost is adopted [36]. XGBoost makes use of the concatenated features, and generates the final classification of the input sentence. It is worth noting that, the upstream feature extraction model is trained also under the task of the sentence classification problem. The reason why upstream model is not directly used for our final task but is used as feature extraction model is because of the following reasons: First, the use of BERT and TextCNN to extract features allows the fusion with other features such as the vector processed by tf-idf or word2vec, thus making a better prediction result possible [37]. Second, the upstream model’s output can be reused for other downstream task, such as hate-speech detection or general sentiment analysis [38] [39]. We collected 354211 sentences under different section of topics from a social media platform Weibo as our dataset, in which each sentence belongs to one of the following classes: Market Monitoring, Finance Industry, Social Welfare, Education Industry, Food Security, Public Security, Crime Case and Medical Health.

Upstream Model We trained BERT and TextCNN by viewing them as sentence classification models and attached a softmax function to transform the features to probability. The loss function we used is multi-class cross entropy. We assume that the upstream model trained using TextCNN is denoted as u , and the upstream model trained using BERT is denoted as u' . After training these two versions of models as the classification task, the accuracy of each model on the validation set is shown in TABLE 4.3.

Table 4.3: The feature extraction quality as classification task before and after the update

Class	TextCNN / BERT	
	original model u	retrained model u'
Market Monitoring	0.870	0.865
Finance Industry	0.872	0.876
Social Welfare	0.861	0.867
Education Industry	0.884	0.868
Food Security	0.856	0.868
Public Security	0.874	0.899
Crime Case	0.864	0.872
Medical Health	0.876	0.880
All Classes	0.869	0.873

It can be observed that compared to the TextCNN model, the retrained upstream BERT model indeed increases in performance overall, as the accuracy of the classification improves from 0.869 to 0.873.

Downstream Model We deleted the softmax layer for classification for upstream TextCNN model u and BERT model u' , and used the remained final layer as the extracted feature. By concatenating

with the same features directly processed from the original data using tf-idf and other statistical information such as the number norms in the sentence, we processed two versions of the extracted features from the sentences. We then use the features to train two versions of the downstream models, which are both XGBoost. Besides the input data, there is no change in training algorithm. The result is shown in Table. 4.4.

Table 4.4: The accuracy of classification before and after the update

XGBoost		
Class	original model u	retrained model u'
Market Monitoring	0.876	0.887
Finance Industry	0.917	0.919
Social Welfare	0.908	0.899
Education Industry	0.960	0.968
Food Security	0.899	0.907
Public Security	0.922	0.938
Crime Case	0.917	0.860
Medical Health	0.972	0.971
All Classes	0.924	0.921

It is shown that the component entanglement improvement indeed happens, as the downstream model using the feature extracted by the original model u of TextCNN has an accuracy of 0.924, while using the updated model u' of BERT results in a decrease of accuracy to 0.921. After the retraining, the upstream model provides extracted features that are supposedly better in classification task. But after the feature fusion and the process of XGBoost, the downstream model’s result deteriorates.

Result Analysis The experiment replicates the complete process of updating an upstream model in a sentence classification setting. The result again confirms the phenomenon of the component entanglement indeed can happen in a two-component MLS when upstream model is retrained. In the proposed system, we retrained the upstream model of feature extraction and its classification accuracy was increased on validation set. However, the downstream model for the actual classification shows a decrease in accuracy, thus showcasing the threat of entangled enhancement in the context of sentence classification task.

Chapter 5

Progressive and Conservative Retraining Policy

5.1 Motivation

As shown in Chapter 4, MLS in different application scenarios faces the threat of entangled enhancement. This phenomenon occurs when retraining appears to improve the performance of a component but ultimately worsens the overall system performance. One of the practical solutions to mitigate the issue of entangled enhancement is the *maintenance policy* that guides every retraining attempt, such as when to retrain upstream or downstream models. By tailoring retraining policies to the specific system, it is possible to maximize service availability. Availability is a system performance metric which provides insight into the coverage factor that an item or system will be available to be committed to a specified requirement [40]. In this chapter, we regard that the system is available when the downstream model's performance on the dataset X_{real} meets the required threshold τ_d . Otherwise, the system is regarded as unavailable. We leverage CTMC modeling to capture the state transitions of the MLS and quantitatively evaluate the service availability by the probability that MLS is in available states in the operation.

To maximize the service availability by effective retraining of ML models, we propose two model retraining policies, the *progressive retraining* policy and the *conservative retraining* policy. The progressive retraining policy reflects the strategy of actively tuning the system to keep the system updated, while the conservative retraining policy attempts retraining only when it is necessary. A comparison between these two policies is conducted through numerical analysis to analyze the effect of these policies on the service availability.

5.2 State Space Modeling

5.2.1 Definition

We model the MLS by a CTMC $\{X(t) \mid t \geq 0\}$. In reference to the performance thresholds, we define the state of the two-component MLS as a pair $(s(u, \tau_u), s(d, \tau_d))$ where

$$s(x, \tau_x) = \begin{cases} 1, & \text{if model } x \text{ satisfies threshold } \tau_x, \\ 0, & \text{otherwise.} \end{cases} \quad (5.1)$$

We assume that the initial state of the system is $(0, 0)$, representing that both upstream model u and downstream model d meet the performance thresholds (i.e., τ_u and τ_d , respectively).

5.2.2 Progressive Retraining Policy

Under this policy, ML models are retrained from time to time to counter the potential performance degradation due to dataset shift. If both upstream model and downstream model satisfy the threshold, the policy performs no retraining is made.

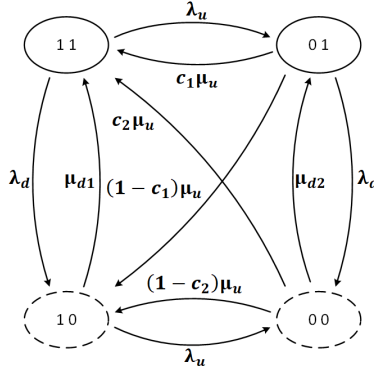


Figure 5.1: A CTMC depicting the dynamic of MLS under progressive retraining policy. The failure rate due to dataset shift for u is λ_u , and for d is λ_d . The recovery rate for u is μ_u , and for d , if it is at the state $(1,0)$, it is μ_{d1} , and if it is at the state $(0,0)$, it is μ_{d2} . c_1 is the coverage factor that at state $(0,1)$, the recovery to u resulted in a transition to state $(1,1)$. c_2 is the coverage factor that at state $(0,0)$, the recovery to u resulted in a transition to state $(1,1)$.

We model the state transitions of a two-component MLS under the progressive retraining policy by the CTMC as shown in Figure 5.1. We assume that the state transition times follow the exponential distributions. We denote the dataset shift transition rate for upstream model u is λ_u , and the dataset shift transition rate for downstream model d is λ_d . The successful recovery rate for upstream model u is μ_u . For downstream model d 's retraining, if the upstream model u is above the threshold, the successful recovery rate is μ_{d1} , and if the upstream model u is below the threshold, the successful recovery rate is μ_{d2} ($> \mu_{d1}$), because recovering the performance of the downstream model is harder when the upstream model is deteriorated. c_1 is the coverage factor for type 1 entangled enhancement. c_2 is the coverage factor for type 2 entangled enhancement.

At state $(1,1)$, no recovery should be observed. Due to the dataset shift, the system might experience compromise in either upstream model u or downstream model d . At state $(0,1)$, the upstream model u fails to reach the threshold while downstream model d remains to be above the threshold. Therefore, only the successful retraining to the upstream model u can be observed. The retraining could be successful, leading a transition back to state $(1,1)$, or it could trigger a entangled enhance-

ment, which transits to state (1,0). At state (0,0), both upstream and downstream models fail to reach the threshold, and hence go under retraining attempts. However, retraining upstream model can have two consequences: 1. the upstream model recovers the performance, while the downstream model fails to recover the performance τ_d . This could be caused by either entangled enhancement, or the downstream model only is improved slightly, which is not enough for the state change. 2. The retraining is successful enough that the downstream model is also benefited from the improvement, so that it reaches to the threshold τ_d . At state (1,0), since only downstream model d is deteriorated, only successful retraining to the downstream model could be observed.

Because each state is reachable from other states, this CTMC is irreducible. For an irreducible CTMC, the state probabilities reach an asymptotic value as the time goes to infinity, and this asymptotic value is independent of the initial condition, and is called steady-state probability. We further denote the steady-state availability of the system as $\pi = \{\pi_i\}$, $i \in \{(0,0), (0,1), (1,0), (1,1)\}$. π can be obtained by solving the system $\pi \cdot Q = 0$ with $\pi e^T = 1$, where Q is the infinitesimal generator matrix of the CTMC, and e^T is the 4-dimension column vector whose elements are 1.

$$Q = \begin{bmatrix} -\lambda_u - \lambda_d & \lambda_u & \lambda_d & 0 \\ c_1\mu_u & -\mu_u - \lambda_d & (1-c_1)\mu_u & \lambda_d \\ \mu_{d1} & 0 & -\lambda_u - \mu_{d1} & \lambda_u \\ c_2\mu_u & \mu_{d2} & (1-c_2)\mu_u & -\mu_u - \mu_{d2} \end{bmatrix}. \quad (5.2)$$

Because the final output of the MLS is the downstream model's output, state (1,1) and (0,1) can be counted as available states. Thus, the service availability of the system under the progressive retraining policy is given by $A_p = \pi_{(1,1)} + \pi_{(0,1)}$.

5.2.3 Conservative Retraining Policy

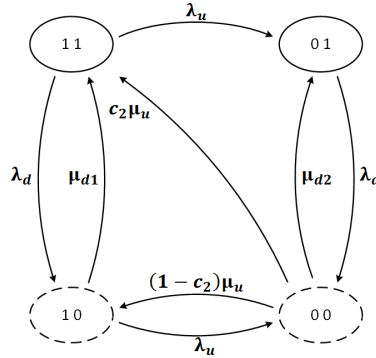


Figure 5.2: A CTMC depicting the dynamic of MLS under conservative retraining policy. The failure rate due to dataset shift for u is λ_u , and for d is λ_d . The recovery rate for u is μ_u , and for d , if it is at the state (1,0), it is μ_{d1} , and if it is at the state (0,0), it is μ_{d2} . c_2 is the coverage factor that at state (0,0), the recovery to u resulted in a transition to state (1,1).

Next, we introduce the conservative retraining policy. Under this policy, ML models are retrained only when observing the deteriorated system performance (i.e., the performance of downstream

model does not satisfy the required threshold τ_d).

The two components MLS under the conservative retraining policy is modeled by the CTMC as shown in Fig. 5.2. At state (1,1) and (0,1), no retraining is attempted because the performance of the downstream model satisfies the threshold. At State (1,0) and (0,0), both upstream and downstream models are being retrained. Compared to the system in Fig. 5.1, the transition from (0,1) to (1,1) is removed. However at the same time, the transition from (0,1) to (1,0), which is the entangled enhancement that causes the system to fail, is also removed, which might benefit the system.

As the CTMC is irreducible, we can compute the steady-state probabilities through the infinitesimal generator:

$$Q = \begin{bmatrix} -\lambda_u - \lambda_d & \lambda_u & \lambda_d & 0 \\ 0 & -\lambda_d & 0 & \lambda_d \\ \mu_{d1} & 0 & -\lambda_u - \mu_{d1} & \lambda_u \\ c_2\mu_u & \mu_{d2} & (1 - c_2)\mu_u & -\mu_u - \mu_{d2} \end{bmatrix}. \quad (5.3)$$

The service availability under the conservative retraining policy can again be derived as $A_p = \pi_{(1,1)} + \pi_{(0,1)}$.

5.3 Numerical Analysis

5.3.1 Research Question

In this section, we perform numerical analyses on the proposed CTMC to investigate the effectiveness of retraining policies in terms of service availability. Specifically, the analysis aims to address the following research question:

- RQ: What factors mostly influence service availability of MLS under progressive and conservative retraining policies?

5.3.2 Parameter Assignment

Table 5.1: Parameter Input

Parameter	Description	Value
$\frac{1}{\lambda_u}$	Mean time for upstream model performance to drop below threshold τ_u due to dataset shift	80 days
$\frac{1}{\lambda_d}$	Mean time for downstream model performance to drop below threshold τ_d due to dataset shift	40 days
$\frac{1}{\mu_u}$	Mean time for upstream model recovery	20 days
$\frac{1}{\mu_{d1}}$	Mean time for downstream model recovery when upstream model satisfies τ_u	10 days
$\frac{1}{\mu_{d2}}$	Mean time for downstream model recovery when upstream model fails to satisfy τ_u	100 days
c_1	Coverage factor for type 1 entangled enhancement	0.75
c_2	Coverage factor for type 2 entangled enhancement	0.5

Table 5.1 presents the input parameter values employed in the analysis. Due to a lack of literature addressing the temporal distribution of dataset shift in real-world scenarios, these values are selected based on assumptions for an MLS operation. Specifically, the mean time to upstream model failure

Table 5.2: Scaled Sensitivity for A_p and A_c

Parameter θ	$SS_{\theta}(A_p)$	$SS_{\theta}(A_c)$
λ_u	-0.0602	-0.0602
λ_d	-0.229	-0.233
μ_u	0.0802	0.122
μ_{d1}	0.194	0.149
μ_{d2}	0.0147	0.0218
c_1	0.0165	-
c_2	0.0238	0.0411

attributable to dataset shift is set to 80 days, and the mean time to upstream model recovery is set to 20 days. For the downstream model, the mean time to failure is set to 20 days, and the mean time for downstream model recovery, triggered when the upstream model reaches the threshold τ_u , is set to 10 days. The mean time for downstream model recovery when the upstream model fails to reach τ_u is configured as 100 days, reflecting the anticipation that such events are unlikely to occur. The coverage factor of an upstream model recovery leading to no downstream model failure when downstream model is already available (denoted as c_1) is set to 0.75. Similarly, the converge factor of an upstream model recovery resulting in the downstream model reaching τ_d and upstream model reaching τ_u (denoted as c_2) is set to 0.5.

5.3.3 Sensitivity Analysis

Tables 5.2 shows the sensitivities of transition rates and coverage factors influencing A_p and A_c . The two most significant transition rates influencing service availability of the system is λ_d , followed by μ_{d1} . Obviously, the recovery rate and coverage factor positively impacts service availability, whereas an increase in the failure rate negatively affects service availability.

We plot A_p and A_c by varying the values of individual parameters as shown in Fig. 5.3. The trends exhibited by the lines in the figures mostly align with the parameters shown in the tables. Fig. 5.3 (a) and (b) show the sensitivity to the dataset transition rate. In Fig. 5.3 (b), there is a changing point where the service availability achieved by the progressive retraining policy overcomes the conservative retraining policy. This phenomenon can also be observed in Fig. 5.3 (c) and (d), which present the sensitivity to the downstream model recovery rate at (1,0) and (0,0), respectively. As shown in the figures, when μ_{d1} becomes large enough, service availability under the progressive retraining policy exceeds the one under the conservative retraining policy, while μ_{d2} shows the completely opposite trend, as when it becomes large enough, service availability under the conservative retraining policy starts to win over the progressive retraining policy. This can be attributed to the fact that under the progressive retraining policy, system is more likely to experience deterioration due to the entangled enhancement represented by the transition from (0,1) to (1,0). μ_{d1} , as the recovery rate from (1,0) to (1,1), can possibly revert the bad influence of entangled enhancement, thus compensating the progressive retraining policy more. On the other hand, μ_{d2} is the recovery rate from (0,0) to (0,1), which would benefit service availability under the conservative retraining policy more, as it is more likely to visit state (0,0) as a failure state. Fig. 5.3 (e) shows

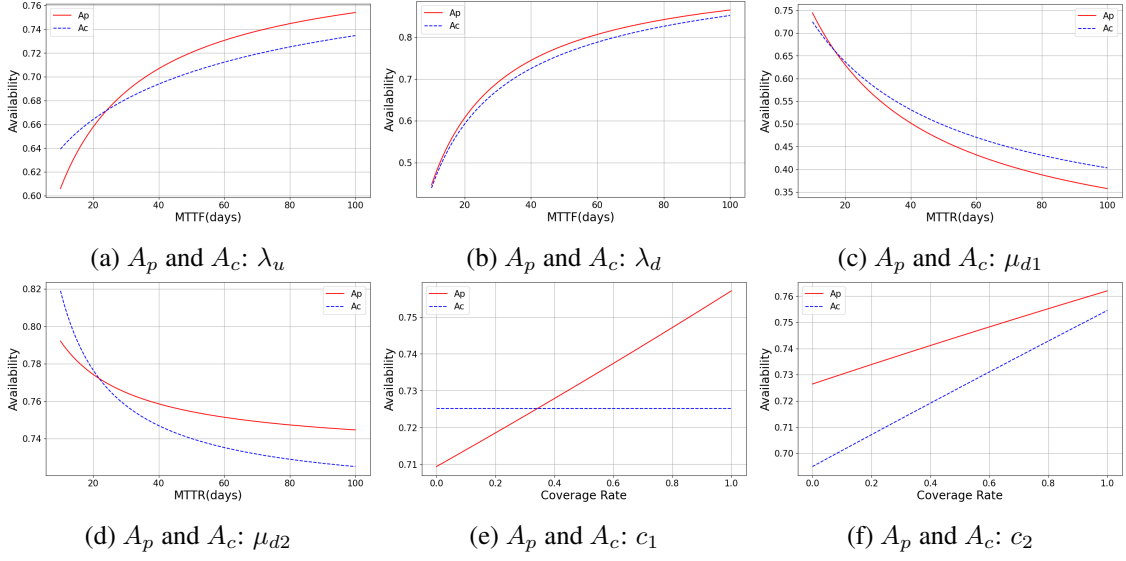


Figure 5.3: Service Availability regarding to different parameter changes

the sensitivity to the coverage factor for transition (0,1) to (1,1). When c_1 is sufficiently small, the progressive retraining policy becomes worse than the conservative policy due to potential negative impacts of the entangled enhancement. In summary, the efficacy of a system's retraining policies in terms of service availability is notably influenced by λ_u , μ_{d1} , μ_{d2} and c_1 , which play pivotal roles in determining the policy that yields a more available system.

5.4 Findings

The result of numerical analysis indicate that, concerning service availability, both policies exhibit advantages under distinct circumstances. Regardless of the policy, the failure rate of the downstream model is of greatest significance, and hence reducing the failure rate is the primary consideration for improving availability.

Chapter 6

Trade-off between Progressive and Conservative Retraining Policy

6.1 Motivation

In Chapter 5, we used CTMC to model the system under different retraining policies. However, different retraining policies affect not only service availability but also other system aspects. Frequent retraining can improve service availability, although it may harm the continuity of service by service interruptions. A higher availability can be achieved at the cost of service continuity. Thus, we need to investigate the potential trade-off between availability and continuity of MLS.

In order to analyze the service availability and continuity under different maintenance policies, we can resort to state-space modeling and analysis as adopted in Chapter 5. However, modeling using CTMC, which assumes the Markov property throughout the retraining and failure process, is overly restrictive. It is unlikely all the retraining times follow the exponential distributions. To overcome the limitation, we leverage SMP to model the state transitions of the MLS. In the following sections, we first give the definition of SMP for the two-component MLS, and proposes SMP models for two maintenance policies, which are progressive retraining and conservative retraining.

6.2 State Space Modeling

6.2.1 Definition

We model the MLS by a SMP $\{X(t) \mid t \geq 0\}$. In reference to the performance thresholds, we define the state of the two-component MLS using Equation 5.1r. We again assume that the initial state of the system is $(0, 0)$, representing that both upstream model u and downstream model d meet the performance thresholds (i.e., τ_u and τ_d , respectively). Additionally, we denote $(0, 0')$ and $(0, 1')$ as vanishing states representing that the state $(0, 0)$ and $(0, 1)$ are changed due to retraining, and are about to transits to other state immediately. We use the kernel function to define our SMP,

$$k_{ij}(t) = P \{X_{n+1} = j, T_{n+1} - T_n \leq t \mid X_n = i\}, \quad (6.1)$$

as the conditional probability that, given that the process has entered state i at time T_n , the next transition occurs at time t toward state j . We collect all the $k_{ij}(t)$ into a matrix, called the kernel matrix of SMP:

$$\mathbf{K}(t) = [k_{ij}(t)]. \quad (6.2)$$

We assume that the embedded discrete-time Markov chain (DTMC) of the SMP satisfies the homogeneity property and hence it is fully characterized by its one-step transition probability matrix (TPM) $\mathbf{P} = [p_{ij}]$, where p_{ij} is the probability that the next jump of the DTMC is to state j given that the process is in state i . We define $H_i(t)$ to be the sojourn time distribution in state i . Specifically,

$$p_{ij} = \lim_{t \rightarrow \infty} k_{ij}(t), \quad \mathbf{P} = \lim_{t \rightarrow \infty} \mathbf{K}(t), \quad H_i(t) = \sum_j k_{ij}(t). \quad (6.3)$$

6.2.2 Progressive Retraining Policy

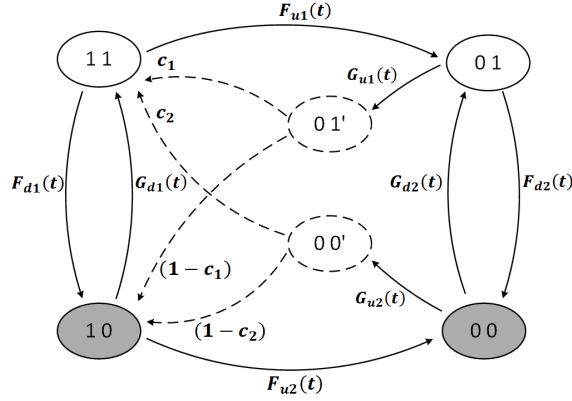


Figure 6.1: A SMP depicting the dynamic of MLS under progressive retraining policy. The sojourn time of the transitions caused by dataset shift is set to follow general functions $F_{u1}(t)$, $F_{u2}(t)$, $F_{d1}(t)$ and $F_{d2}(t)$. The sojourn time of the transitions caused by retraining is set to follow some general distribution function. $G_{u1}(t)$ denotes the sojourn time distribution from $(0, 1)$ to $(0, 1')$. $G_{u2}(t)$ denotes the sojourn time distribution from $(0, 0)$ to $(0, 0')$. $G_{d1}(t)$ denotes the sojourn time distribution from $(1, 0)$ to $(1, 1)$. $G_{d2}(t)$ denotes the sojourn time distribution from $(0, 0)$ to $(0, 1)$. $(0, 1')$ and $(0, 0')$ are vanishing states with sojourn time of 0. c_1 denotes the coverage factor that once $(0, 1')$ is entered, it will be transited to $(1, 1)$. c_2 denotes the coverage factor that once $(0, 0')$ is entered, it will be transited to $(1, 1)$.

We model the state transitions of a two-component MLS under the progressive retraining policy by the SMP as shown in Figure 6.1. We denote the the sojourn time of state transitions caused by dataset shift, which are $(1, 1) \rightarrow (0, 1)$, $(1, 0) \rightarrow (0, 0)$, $(1, 1) \rightarrow (1, 0)$ and $(0, 1) \rightarrow (0, 0)$, to follow some general distribution $F_{u1}(t)$, $F_{u2}(t)$, $F_{d1}(t)$ and $F_{d2}(t)$ respectively. We denote the the sojourn time of state transitions caused by retraining, which are $(0, 1) \rightarrow (0, 1')$, $(0, 0) \rightarrow (0, 0')$, $(1, 0) \rightarrow (1, 1)$ and $(0, 0) \rightarrow (1, 1)$, to follow some general distribution $G_{u1}(t)$, $G_{u2}(t)$, $G_{d1}(t)$ and

$G_{d2}(t)$ respectively. Vanishing states $(0, 1')$ and $(0, 0')$ has sojourn time of 0. c_1 is the coverage factor for type 1 entangled enhancement. c_2 is the coverage factor for type 2 entangled enhancement.

At state $(1, 1)$, no recovery should be observed. Due to the dataset shift, the system might experience compromise in either upstream model u or downstream model d . At state $(0, 1)$, the upstream model u fails to reach the threshold while downstream model d remains to be above the threshold. Therefore, only the successful retraining to the upstream model u which leads to state $(0, 1')$ can be observed. If the retraining is successful, it leads to a transition from state $(0, 1')$ to state $(1, 1)$, or it could trigger a Type 1 entangled enhancement, which transits from state $(0, 1')$ to state $(1, 0)$. At state $(0, 0)$, both upstream and downstream models fail to reach the threshold, and hence go under retraining attempts, and attempts that change the state of SMP would lead a transition to $(0, 0')$. However, retraining upstream model can have two consequences: 1. the upstream model recovers the performance, while the downstream model fails to recover the performance τ_d . This could be caused by either a Type 2 entangled enhancement, or the downstream model only is improved slightly, which is not enough for the state change. This would make SMP transits to $(1, 0)$. 2. The retraining is successful enough that the downstream model is also benefited from the improvement, so that it reaches to the threshold τ_d . This would make SMP transits to $(1, 1)$. At state $(1, 0)$, only retraining the downstream model can restore it to $(1, 1)$.

In order to derive the steady-state probability of the SMP, we need to obtain the steady-state probability vector $\mathbf{v}$ of the embedded DTMC by solving the linear system of equations $\mathbf{v} = \mathbf{v}\mathbf{P}$ w.r.t $\mathbf{v}\mathbf{e}^T = 1$, where $\mathbf{e}$ is the vector with all entries equal to 1. In SMP, TPM can be deduced by the using the Equation 6.3, therefore we can obtain the following $\mathbf{P}$ matrix:

$$\mathbf{P} = \begin{matrix} & \begin{matrix} (1, 1) & (0, 1) & (1, 0) & (0, 0) & (0, 1') & (0, 0') \end{matrix} \\ \begin{pmatrix} 0 & P_1 & 1 - P_1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 - P_2 & P_2 & 0 \\ P_3 & 0 & 0 & 1 - P_3 & 0 & 0 \\ 0 & P_4 & 0 & 0 & 0 & 1 - P_4 \\ c_1 & 0 & 1 - c_1 & 0 & 0 & 0 \\ c_2 & 0 & 1 - c_2 & 0 & 0 & 0 \end{pmatrix} \end{matrix}, \quad (6.4)$$

$$\begin{cases} P_1 = \int_0^\infty (1 - F_{d1}(t))dF_{u1}(t), \\ P_2 = \int_0^\infty (1 - F_{d2}(t))dG_{u1}(t), \\ P_3 = \int_0^\infty (1 - F_{u2}(t))dG_{d1}(t), \\ P_4 = \int_0^\infty (1 - G_{u2}(t))dG_{d2}(t). \end{cases} \quad (6.5)$$

The steady-state probability $\mathbf{v}$ of the embedded DTMC can be derived. Let $h_i = \int_0^\infty (1 - H_i(t))dt$ be the mean sojourn time in state i , then we have:

$$\begin{cases} h_{(1,1)} = \int_0^\infty (1 - F_{u1}(t))(1 - F_{d1}(t))dt, \\ h_{(0,1)} = \int_0^\infty (1 - G_{u1}(t))(1 - F_{d2}(t))dt, \\ h_{(1,0)} = \int_0^\infty (1 - G_{d1}(t))(1 - F_{u2}(t))dt, \\ h_{(0,0)} = \int_0^\infty (1 - G_{u2}(t))(1 - G_{d2}(t))dt, \\ h_{(0,1')} = h_{(0,0')} = 0. \end{cases} \quad (6.6)$$

The steady-state probability π_i for the SMP is given by

$$\pi_i = \frac{v_i h_i}{\sum_j v_j h_j}. \quad (6.7)$$

6.2.3 Conservative Retraining Policy

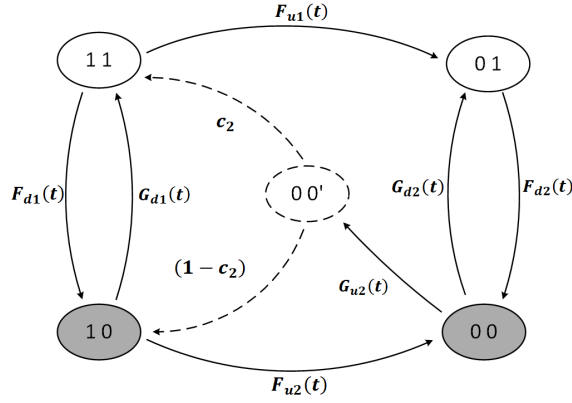


Figure 6.2: A SMP depicting the dynamic of MLS under conservative retraining policy. The sojourn time of the transitions caused by dataset shift is set to follow general functions $F_{u1}(t)$, $F_{u2}(t)$, $F_{d1}(t)$ and $F_{d2}(t)$. The sojourn time of the transitions caused by retraining is also set to follow some general distribution function. $G_{u2}(t)$ denotes the sojourn time distribution from $(0,0)$ to $(0,0')$. $G_{d1}(t)$ denotes the sojourn time distribution from $(1,0)$ to $(1,1)$. $G_{d2}(t)$ denotes the sojourn time distribution from $(0,0)$ to $(0,1)$. $(0,0')$ is a vanishing states with sojourn time of 0. c_2 denotes the possibility that once $(0,0')$ is entered, it will be transited to $(1,1)$.

The two components MLS under the conservative retraining policy is modeled by the SMP as shown in Fig. 6.2. At state $(1,1)$ and $(0,1)$, no retraining is attempted because the performance of the downstream model satisfies the threshold. At State $(1,0)$, the downstream model would be retrained. At State $(0,0)$, both upstream and downstream models could be retrained. Compared to the system in Fig. 6.2, because of there would be no retraining happen in state $(0,1)$, the state $(0,1')$ is also absent. The transition from $(0,1')$ to $(1,1)$ is removed, which is a potential loss. However

at the same time, the transition from (0,1') to (1,0), which is the Type 1 entangle enhancement that causes the system to fail, is also removed, which might benefit the system.

Because the embedded DTMC of the SMP satisfies the homogeneity property, we can obtain the steady-state probability vector $\mathbf{v}$ by $\mathbf{P}$ using the formula $\mathbf{P} = \lim_{t \rightarrow \infty} \mathbf{K}(t)$ again,

$$\mathbf{P} = \begin{pmatrix} (1,1) & (0,1) & (1,0) & (0,0) & (0,0') \\ 0 & P_1 & 1-P_1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ P_2 & 0 & 0 & 1-P_2 & 0 \\ 0 & P_3 & 0 & 0 & 1-P_3 \\ c_2 & 0 & 1-c_2 & 0 & 0 \end{pmatrix}, \quad (6.8)$$

$$\begin{cases} P_1 = \int_0^\infty (1 - F_{d1}(t)) dF_{u1}(t), \\ P_2 = \int_0^\infty (1 - F_{u2}(t)) dG_{d1}(t), \\ P_3 = \int_0^\infty (1 - G_{u2}(t)) dG_{d2}(t). \end{cases} \quad (6.9)$$

After computing the steady-state probability $\mathbf{v}$, we derive the mean sojourn time of state i h_i :

$$\begin{cases} h_{(1,1)} = \int_0^\infty (1 - F_{u1}(t))(1 - F_{d1}(t)) dt, \\ h_{(0,1)} = \int_0^\infty (1 - F_{d2}(t)) dt, \\ h_{(1,0)} = \int_0^\infty (1 - G_{d1}(t))(1 - F_{u2}(t)) dt, \\ h_{(0,0)} = \int_0^\infty (1 - G_{u2}(t))(1 - G_{d2}(t)) dt, \\ h_{(0,0')} = 0. \end{cases} \quad (6.10)$$

Steady-state probability π_i for the SMP state i can be derived from Equation 6.7.

6.2.4 Metrics Definition

To compare the effectiveness of the system under different policies, we define *service availability* and *service continuity* as the evaluation metrics, and provide the formula for the calculation of these metrics.

Specifically, under the policy k , we define U_k as the set that comprise all the states that are deemed as available states, D_k as the set that comprise all the states that are deemed as unavailable states and π_i as the steady-state availability of state i , which can be derived by Equation 6.7.

The service availability under the policy k is defined as:

$$A_k = \prod_{i \in U_k} \pi_i. \quad (6.11)$$

The metric reflects the long-term probability that the system meets user satisfaction by reaching the threshold when the user utilizes the MLS.

The service continuity is defined as the average duration in between transitions happen from an available state $((1,1), (0,1) \text{ or } (0,1'))$ to an unavailable state $((1,0), (0,0) \text{ or } (0,0'))$. The higher the service continuity, the more stable the system under that policy is, as it implies a longer duration in between failure events. Formally, service continuity under policy k can be defined as the inverse of the sum of throughput from the available state set U_k to the unavailable state set D_k . We define $tp_{i,j}$ as the throughput from state i to state j . To calculate $tp_{i,j}$, we first derive the expected duration between system entering state i and system entering state i again, denoted by W_i :

$$W_i = \frac{\sum_k v_k h_k}{v_i}. \quad (6.12)$$

Then the frequency of state i in the long run is exactly $\frac{1}{W_i}$. By multiplying the transition probability of the embedded DTMC from state i to state j , we derive the $tp_{i,j}$:

$$tp_{i,j} = \frac{1}{W_i} * p_{i,j}. \quad (6.13)$$

And the service continuity C_k can be mathematically expressed as:

$$C_k = \frac{1}{\sum_{i \in U_k, j \in D_k} tp_{i,j}}. \quad (6.14)$$

6.3 Numerical Analysis

6.3.1 Research Questions

In this section, we perform numerical analyses on the proposed SMPs to investigate the effectiveness of retraining policies in terms of service availability and service continuity. Specifically, the analysis aims to address the following research questions:

- RQ1: What factors mostly influence service availability and service continuity of MLS under progressive and conservative retraining policies?
- RQ2: How does the shape of the distribution function of the state transition time impact the analysis results?
- RQ3: How is service availability correlated or in trade-off relation with service continuity under different policies?

Based on the analysis, we also present a strategy to choose the right retraining policies for meeting the requirements from the MLS.

6.3.2 Parameter Assignment using Weibull Holding Time

Table 6.1 and Table 6.2 present the input parameter functions and values used in the analysis. Since SMP allows general distributions for all the state transition times, we employ the Weibull

Table 6.1: Sojourn Time Distribution

Sojourn time distribution	Function or value	Transition states
$F_{u1}(t)$	$1 - \exp[-(\eta_1 t)^1]$	$(1, 1) \rightarrow (0, 1)$
$F_{u2}(t)$	$1 - \exp[-(\eta_1 t)^1]$	$(1, 0) \rightarrow (0, 0)$
$F_{d1}(t)$	$1 - \exp[-(\eta_2 t)^1]$	$(1, 1) \rightarrow (1, 0)$
$F_{d2}(t)$	$1 - \exp[-(\eta_2 t)^1]$	$(0, 1) \rightarrow (0, 0)$
$G_{u1}(t)$	$1 - \exp[-(\eta_3 t)^{\beta_1}]$	$(0, 1) \rightarrow (0, 1')$
$G_{u2}(t)$	$1 - \exp[-(\eta_3 t)^{\beta_1}]$	$(0, 0) \rightarrow (0, 0')$
$G_{d1}(t)$	$1 - \exp[-(\eta_4 t)^{\beta_2}]$	$(1, 0) \rightarrow (1, 1)$
$G_{d2}(t)$	$1 - \exp[-(\eta_5 t)^{\beta_3}]$	$(0, 0) \rightarrow (0, 1)$

Table 6.2: Parameter Input

Parameter	Value	Description
η_1	0.0125	Scale parameter of $F_{u1}(t)$, $F_{u2}(t)$
η_2	0.025	Scale parameter of $F_{d1}(t)$, $F_{d2}(t)$
η_3	0.057	Scale parameter of $G_{u1}(t)$, $G_{u2}(t)$
η_4	0.112	Scale parameter of $G_{d1}(t)$
η_5	0.011	Scale parameter of $G_{d2}(t)$
β_1	2	Shape parameter of $G_{u1}(t)$, $G_{u2}(t)$
β_2	3	Shape parameter of $G_{d1}(t)$
β_3	3	Shape parameter of $G_{d2}(t)$
c_1	0.75	Coverage factor for type 1 entangled enhancement
c_2	0.5	Coverage factor for type 2 entangled enhancement

distribution as it can describe the transition behavior with increasing, constant, and decreasing hazard rates. We use 2-parameter Weibull distributions to model the sojourn time distribution, which consists of 2 parameters: the scale parameter η and the shape parameter β . The scale parameter η locates the position of the probability density function (PDF) of the distribution on the time axis. As η increases, the entire distribution shifts to the left, which means that the time to state transition become shorter. For a larger η , recoveries or failures are expected to happen more quickly. The shape parameter β determines the shape of the PDF and is closely related to the hazard rate of the distribution. When $\beta < 1$, the hazard rate is monotonically decreasing, and when $\beta > 1$, the hazard rate is monotonically increasing. As β increases from 1 to higher values, the peak of the PDF becomes more pronounced and shifts to the right. This would generally increase the expected time to state transition, but at the same time makes such transition more deterministic and predictable, as the recovery/failure time cluster around a certain period.

The parameter values in Table VI are chosen from the parameter space satisfying the following conditions:

$$\eta_1 < \eta_2, \quad (6.15)$$

Table 6.3: Scaled Sensitivity for Availability and Continuity

Parameter θ	$SS_\theta(A_p)$	$SS_\theta(A_c)$	$SS_\theta(C_p)$	$SS_\theta(C_c)$
η_1	-0.0643	-0.0601	0.00383	0.0601
η_2	-0.201	-0.205	-0.700	-0.787
η_3	0.0811	0.166	-0.112	-0.166
η_4	0.181	0.0956	-0.181	-0.0955
η_5	-0.000900	0.00205	-0.00182	-0.00205
β_1	-0.000302	0.00168	0.00303	-0.00168
β_2	-0.0102	-0.00625	0.0102	0.00625
β_3	-0.00125	-0.00286	0.00241	0.00286
c_1	0.0413	-	0.169	-
c_2	0.0212	0.0253	-0.0212	-0.0253

$$\int_0^\infty \exp[-(\eta_4 t)^{\beta_2}] d(t) < \int_0^\infty \exp[-(\eta_3 t)^{\beta_1}] d(t), \quad (6.16)$$

$$\int_0^\infty \exp[-(\eta_3 t)^{\beta_1}] d(t) < \int_0^\infty \exp[-(\eta_5 t)^{\beta_3}] d(t), \quad (6.17)$$

$$c_1 > c_2. \quad (6.18)$$

For condition 6.15 and condition 6.16, given the cascading effect of upstream model recoveries on all downstream models, the upstream model is presumed to exhibit greater robustness against dataset shift and consequently undergoes recoveries less frequently. Therefore, the mean time to failure (MTTF) and the mean time to recovery (MTTR) of the upstream model should be larger than the MTTF and MTTR of the downstream model.

For condition 6.17, the direct update of downstream model resulting in a restoration in system performance while the upstream model still maintaining a deteriorated performance is very unlikely. Therefore, the MTTR of this process is the largest compared to other recovery process.

For condition 6.18, c_2 represents the possibility of a more demanding condition, requiring the upstream model's recovery making the downstream model's performance metric to improve from below τ_d to above τ_d . In comparison, c_1 only stands for the possibility that the upstream model's recovery does not negatively affect the downstream model to a point that it fails to reach the threshold, which is a condition more prone to happen. Therefore, c_1 is likely to be larger than c_2 .

6.3.3 RQ1: Sensitivity Analysis

The first two columns in Tables 6.3 show the scaled sensitivities of different parameters influencing A_p and A_c , which are the service availability of the system under progressive retraining policy and conservative retraining policy respectively. For both A_p and A_c , the three most important parameters are η_2 , η_4 and η_3 , which represent the scale parameters of the Weibull sojourn time of the transactions due to downstream model dataset shift, downstream model recovery, and upstream model recovery respectively. It is obvious that scale parameter related to distribution of dataset shift

negatively related with availability, while the scale parameter related to distribution of recovery positively related with availability.

The last two columns in Table 6.3 show the sensitivities of various parameters influencing C_p and C_c , which are the service continuity of the system under progressive retraining policy and conservative retraining policy respectively. The three most important parameters affecting service continuity are again η_2 , η_3 , and η_4 . An interesting phenomenon is that, most parameters have opposite effects on availability and continuity. For example, increasing η_3 and η_4 would increase the availability at the cost of continuity regardless of the policy choice. This finding implies that increasing the frequency of the model retraining could improve availability but reduces the continuity.

Findings: Reducing downstream model degradation is the most effective method to increase both availability and continuity. Besides, increasing the frequency of model retraining to gain more service availability has the potential risk of harming service continuity.

6.3.4 RQ2: Comparison to CTMC

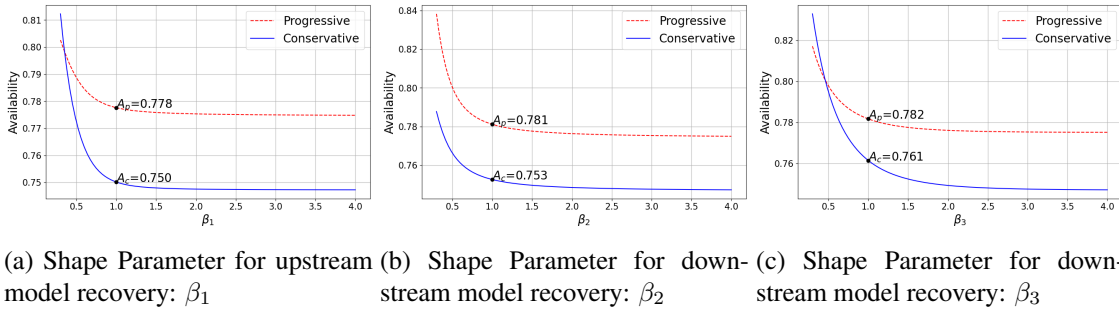


Figure 6.3: Service Availability regarding to different shape parameters

In the previous chapter, we used CTMC for availability analysis. The same analysis can be replicated using SMP by setting the shape parameter of the Weibull distribution of the recovery process to 1, making the transition time exponentially distributed. However, in reality, recovery times may follow various distributions. To demonstrate the impact of different distribution assumptions, we vary the shape parameter of the Weibull distribution while keeping the MTTR constant. Figure 6.3 illustrates how the shape parameter affects service availability.

The figures show that even with a fixed MTTR, increasing the shape parameter β for the recovery process results in a decrease in availability. This can be explained by the behavior of the PDF of the distribution: when β is small, the PDF is more concentrated on the left, indicating a higher likelihood of quick recovery. As β increases, the PDF spreads out and shifts to the right, leading to a more consistent recovery time, thus possibly reducing service availability.

Furthermore, an interesting trend is observed in Figure 6.3 (a) and (c). Initially, the progressive retraining policy shows lower availability compared to the conservative retraining policy. However, as the shape parameter increases, the progressive policy eventually surpasses the conservative policy in terms of availability.

These observations cannot be captured by CTMC modeling, where the hazard rate remains constant (with a shape parameter equal to 1), as depicted by the two points on each figure. Therefore, the previous model had a shortcoming in accurately estimating availability due to the exponential assumption. In contrast, the proposed SMP can accommodate diverse recovery time distributions with different shapes, making a more comprehensive analysis to MLS possible.

Findings: Despite the same MTTR, shape of PDF for recovery process would influence the service availability and even the preference in policy choice. In this aspect, SMP provides a more comprehensive analysis than CTMC.

6.3.5 RQ3: Trade-off analysis: Availability vs Continuity

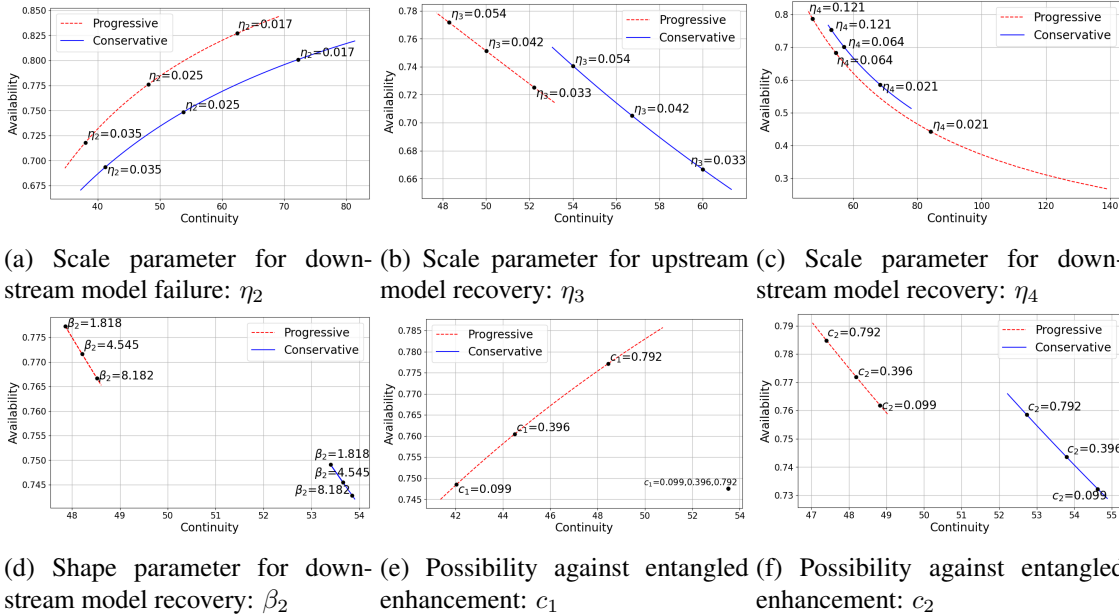


Figure 6.4: Service Availability and Service Continuity regarding to different parameter changes

In order to analyze the trade-off between availability and continuity, we further plot service availability against service continuity under different retraining policies by varying individual parameters, as shown in Fig. 6.4. In Fig. 6.4 (a), increasing the scale parameter for downstream model failure reduces both service availability and service continuity, regardless of the policy. Figures 6.4 (b) and (c) illustrate the relationship between service availability and the scale parameters η_3 for upstream model recovery and η_4 for downstream model recovery (when only the downstream model fails while the upstream model works). An increase in these parameters enhances service availability but decreases service continuity. This aligns with the results in Table 6.1.

Further examination from the figures reveals a mostly consistent trend wherein the system operating under the progressive retraining policy exhibits higher availability compared to conservative

retraining policy. At the same time, the system operating under conservative retraining policy consistently exhibits a higher continuity in comparison to the system employing progressive retraining policy given the same parameters. The result implies that, under the progressive retraining policy, the system on average is more available than the system under conservative retraining policy. On the other hand, under the conservative retraining policy, transitions from an available state to an unavailable state occur less frequently. Consequently, there is a trade-off between service availability and service continuity in choosing different policies. When prioritizing service continuity, the conservative retraining policy is deemed more favorable. In order to gain more service availability, progressive retraining policy should be chosen.

Findings: The maintenance policies can confront the significant trade-off between service availability and service continuity. Therefore, choosing one policy typically enhances one aspect at the expense of the other.

6.4 Strategy Analysis

The trade-off analysis results can guide the maintenance of the MLS that contains strategy to mitigate the imperfect retraining. We found that in most case, the MLS faces the trade-off between service availability and service continuity. Given a specific use case of the MLS, either service availability or service continuity could be more vital compared to the other metric. For example, ML tasks that prioritize high long-term average accuracy, while tolerating occasional performance drops, should focus on service availability rather than service continuity. Predictive maintenance is one key example of such a task. It is a continuous process that requires regular updates and refinements to adapt to changing equipment and evolving operational conditions [41]. Industry practices in predictive maintenance emphasize improving the interpretability of anomaly detection for unsupervised methods and generating more labeled data for supervised methods to enhance long-term accuracy. This approach prioritizes building robust models that perform well over extended periods, rather than maintaining stability during specific short-term intervals [11]. Therefore, ML engineers working on predictive maintenance need a higher service availability. Conversely, some ML tasks demand high run-time accuracy, where model updates that could compromise performance should be avoided. In these scenarios, service continuity is more critical. One example is ML systems for patient care. While such systems can benefit from periodic updates using data collected from patients to improve diagnostic accuracy or support better management decisions [42], frequent retraining poses risks [43]. In such cases, service continuity tends to be prioritized to ensure consistent and reliable real-time performance, safeguarding patient trust and safety.

To ensure the MLS effectively achieves its goal, it is crucial to select the right retraining policy. From the trade-off analysis in Figure 6.4 (e), we observe that when the possibility of entangled enhancement becomes very large, which is represented by the value of c_1 approaches to 0 (for example, $c_1 = 0.1$), the progressive retraining policy should not be applied as it is inferior to conservative retraining policy in both service availability and service continuity. On the other hand, from Figure 6.4 (c), it can be observed that if the successful retraining of the downstream model happens rarely (for example, $\eta_4 = 0.06$), the conservative retraining policy should be applied.

Based on these observations, the system engineers may derive a simple strategy as presented in Figure 6.5.

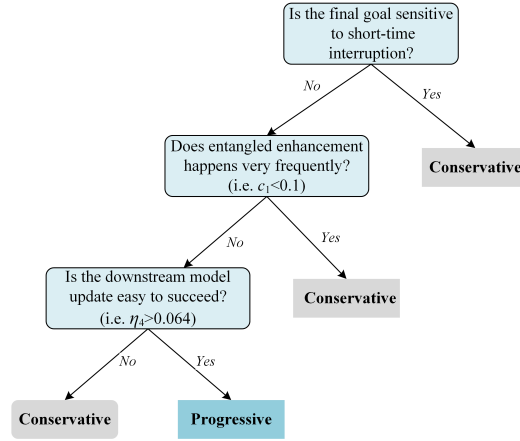


Figure 6.5: A strategy to select the retraining policy for the MLS

Although, the system parameter values differ in other application scenarios, the strategy to choose the retraining policy can be applied in other scenarios as well.

Chapter 7

Dynamic Retraining Policy

7.1 Motivation

In Chapter 5, we introduced two distinct retraining strategies: the progressive retraining policy and the conservative retraining policy. The progressive retraining policy involves continuous model retraining to proactively address potential performance degradation due to dataset shifts. In contrast, the conservative retraining policy initiates model retraining only when the system manager detects a decline in performance.

The previous numerical analysis in Chapter 6 highlights a trade-off between these two approaches. The conservative retraining policy consistently excels in maintaining service continuity, ensuring stable real-time performance. On the other hand, the progressive retraining policy generally achieves higher service availability across most parameter ranges by continuously updating the model to adapt to evolving conditions. This trade-off arises because the progressive retraining policy has a higher likelihood of introducing entanglement when the upstream model fails to meet performance thresholds, even if the downstream model remains functional. In contrast, the conservative retraining policy mitigates this risk by preserving the performance of the upstream model, although it may do so at the expense of service availability.

Given these trade-offs, we propose a *dynamic retraining* policy that flexibly integrates the strengths of the progressive and conservative retraining policies. The dynamic retraining policy allows selected retraining of the upstream model by using a monitoring mechanism during the working cycle of the MLS. Specifically, when the retraining passes the monitoring mechanism and is deemed as a good retraining attempt, the system transitions to a progressive retraining mode when retraining is deemed necessary to enhance service availability. If the retraining failed to pass the monitoring mechanism, it would be canceled and the system reverts to a conservative retraining mode to maintain service continuity. Thereby, both service availability and continuity are maintained in a desirable manner.

The relative importance of the two metrics, service availability and continuity, depends on the specific use case of the MLS. As discussed in Chapter 6, tasks prioritizing long-term accuracy while tolerating occasional performance drops should emphasize service availability. Conversely, tasks requiring high real-time accuracy, where model updates could compromise performance, must prioritize service continuity. The dynamic retraining policy provides a tailored approach, allowing

MLS designers to balance service availability and continuity based on the specific requirements of their application. This ensures maximum utility in managing the MLS effectively.

7.2 Mechanism of Dynamic Retraining

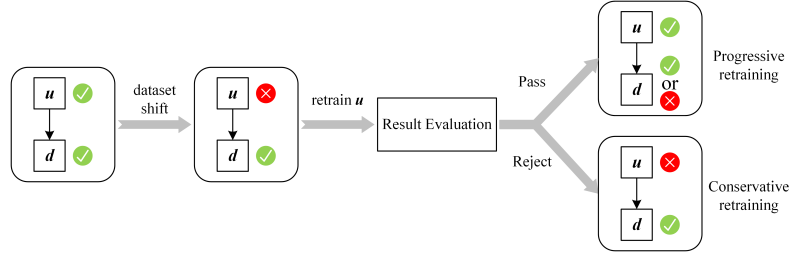


Figure 7.1: Mechanism for dynamic retraining policy. When the upstream model failure is detected, an upstream model retraining would be attempted. The result of the retrained model would be evaluated. Based on the evaluation standard and the retraining result, the upstream model could be updated with the retrained model, or the retraining would be canceled.

The mechanism for the dynamic retraining policy is illustrated in Figure 7.1. The dynamic retraining policy incorporates a monitoring mechanism to evaluate the necessity and efficacy of retraining the upstream model. Upon detecting a failure in the upstream model, a retraining process is initiated. The retrained model is then evaluated against a predefined metric to determine its suitability. This evaluation can result in two possible outcomes: if the retrained model satisfies the established metric, it is adopted as the new upstream model, effectively implementing a progressive retraining policy for that iteration. Conversely, if the retrained model fails to meet the required standard, the retraining effort is discarded, and the system retains the existing model, thereby aligning with a conservative retraining policy for that cycle.

A key design issue of the proposed dynamic approach is the selection of the predefined metric to decide whether retraining should proceed. Ideally, this metric should assess the risk of entanglement arising from upstream model retraining. Specifically, the metric should help identify scenarios where retraining the upstream model could inadvertently trigger entangled enhancement, negatively impacting system performance. When the risk of entanglement is high, the retraining process should be aborted to maintain system stability. Conversely, when the risk is low, retraining can proceed. By incorporating this filtering mechanism into the retraining process, the dynamic retraining policy aims to mitigate the likelihood of entangled enhancement, thereby enhancing the overall stability and reliability of the system.

We propose two potential metrics that could be used for the decision of retraining policy.

Weighted Accuracy in Critical Regions (WACR) This metric is designed to assess the accuracy of the upstream model within the particular data space that holds the most significance for the downstream model. In calculating WACR, greater weight is assigned to instances within these

critical regions, thereby emphasizing their importance. Formally, WACR is defined as:

$$\text{WACR} = \frac{\sum_{i \in \text{critical}} w_i \cdot 1(y_i = \hat{y}_i)}{\sum_{i \in \text{critical}} w_i} \quad (7.1)$$

Here, $1(y_i = \hat{y}_i)$ is an indicator function that equals 1 if the prediction is correct and 0 otherwise, and w_i represents the weight assigned to each instance i within the critical region. The higher the WACR is, the more likely the retraining attempt is going to be accepted.

WACR is particularly useful in scenarios where retraining the upstream model enhances overall accuracy, but the most significant improvements occur in areas of the data space that do not benefit the downstream model, thereby leading to entangled enhancement. An example of such scenario can be observed in experiments involving the Pseudo-Lidar system for 3D object detection in autonomous driving in Chapter 4. As demonstrated in Table 4.1, following retraining, the upstream model exhibits improved performance in estimating disparities for short-range objects, while its ability to accurately estimate disparities for objects at greater distances is compromised. Given that vehicles are generally positioned farther from the camera, this shift in model proficiency does not translate into proportional benefits for the downstream model, ultimately resulting in a paradoxical degradation of overall performance. By utilizing WACR and assigning higher weights to objects located more than 20 meters away, the impact of retraining on the downstream model can be more accurately assessed, providing a clearer understanding of its effects.

Regret Ratio (RR) This metric quantifies the proportion of previously correct predictions that become incorrect after the model is updated, relative to the overall improvement in performance. WACR can sometimes be impractical, particularly when it is challenging to identify the critical data space that significantly influences the downstream model’s performance. In such cases, an alternative approach is to assess whether the retraining of the upstream model introduces excessive new errors, thereby posing a risk. Even if the upstream model shows overall improvement, the retraining attempt can be reconsidered if it leads to a substantial number of previously accurate predictions becoming incorrect.

Formally, the Regret Ratio is defined as:

$$RR = \frac{\text{New Errors}}{\text{Fixed Errors}} \quad (7.2)$$

A high RR indicates that the model’s improvements are accompanied by a significant drawback, where many correct predictions are being compromised. RR is a versatile metric applicable to any MLS with a supervised learning upstream model. Unlike the WACR, RR is particularly useful when all data points hold approximately equal importance in determining the final outcome. For instance, in the sentence classification task described in Chapter 4, the experiment results in Table 4.3 illustrate the impact of retraining: while classification accuracy improved for most classes, it deteriorated significantly for Market Monitoring and the Education Industry. By using RR as a threshold, the risks associated with such retraining can be effectively identified, enabling the prevention of updates that could lead to undesirable outcomes. However, RR does have a limitation in that it does not account for the interdependencies between the upstream and downstream models.

7.3 State Space Modeling

7.3.1 Definition

The definition of the SMP process and the state definition follows from Chapter 6. We also denote T to be the event that the retraining passes the test under dynamic retraining policy. We further denote S to be the event that the retraining being implemented is going to restore the downstream model, and F to be the event that the retraining being implemented triggered entangled enhancement.

7.3.2 Dynamic Retraining Policy

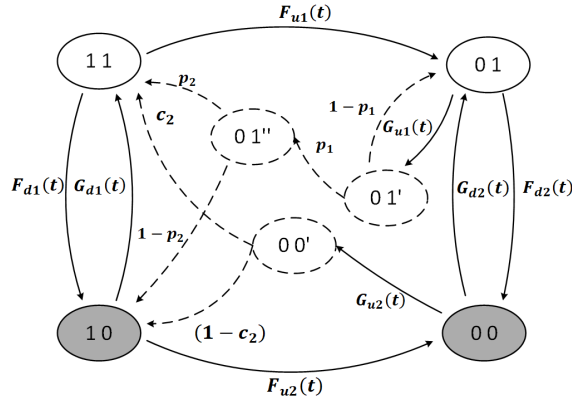


Figure 7.2: A SMP depicting the dynamic of MLS under dynamic retraining policy. The sojourn time of the transitions caused by dataset shift is set to follow general functions $F_{u1}(t)$, $F_{u2}(t)$, $F_{d1}(t)$ and $F_{d2}(t)$. $G_{u1}(t)$ denotes the sojourn time distribution from $(0, 1)$ to $(0, 1')$. $G_{u2}(t)$ denotes the sojourn time distribution from $(0, 0)$ to $(0, 0')$. $G_{d1}(t)$ denotes the sojourn time distribution from $(1, 0)$ to $(1, 1)$. $G_{d2}(t)$ denotes the sojourn time distribution from $(0, 0)$ to $(0, 1)$. $(0, 1')$, $(0, 1'')$ and $(0, 0')$ are vanishing states with sojourn time of 0. p_1 denotes the possibility that once $(0, 1')$ is entered, it will be transited to $(0, 1'')$, which means the retraining is accepted and updated. p_2 denotes the possibility that once $(0, 1'')$ is entered, it will be transited to $(1, 1)$. c_2 denotes the coverage factor for type 2 entangled enhancement.

The two components MLS under the dynamic retraining policy is modeled by the SMP as shown in Fig. 7.2. Compared to MLS under the progressive retraining policy and conservative retraining policy, the main difference happens at state $(0, 1)$. At State $(0, 1)$, upstream model is deteriorated while downstream model still meets the threshold. A retraining attempt would cause the transition to happen from $(0, 1)$ to $(0, 1')$. At State $(0, 1')$, evaluation process would be initiated. We denote the probability of the result of the retraining being accepted and updated to be p_1 . We expect that after the evaluation process, the possibility of entangled enhancement happening would decrease, therefore we use p_1 to denote this probability instead of c_1 , which is the probability of entangled enhancement under progressive retraining policy.

Formally, we calculate p_1 and p_2 , which are the possibility of transition from state $(0, 1')$ to

$(0, 1'')$, and possibility of transition from state $(0, 1'')$ to $(1, 1)$ formally.

$$p_1 = P(T | S) \cdot c_1 + P(T | F) \cdot (1 - c_1), \quad (7.3)$$

$$p_2 = \frac{P(T | S) \cdot c_1}{P(T | S) \cdot c_1 + P(T | F) \cdot (1 - c_1)}. \quad (7.4)$$

Here, $P(T | S)$ and $P(T | F)$ denotes the conditional probability that the retraining passes the evaluation process given that it is a successful retraining that improve the downstream model, and given that it is a retraining that causes entangled enhancement, respectively. These two probabilities reflects the reliability of the evaluation metric. They can also be viewed as the true positive and false positive rates of the thresholds in identifying the successful retraining.

After deducing p_1 and p_2 , we can obtain the following $\mathbf{P}$ matrix:

$$\mathbf{P} = \begin{matrix} & \begin{matrix} (1, 1) & (0, 1) & (1, 0) & (0, 0) & (0, 1') & (0, 0') & (0, 1'') \end{matrix} \\ \begin{matrix} (1, 1) \\ (0, 1) \\ (1, 0) \\ (0, 0) \\ (0, 1') \\ (0, 0') \\ (0, 1'') \end{matrix} & \begin{pmatrix} 0 & P_1 & 1 - P_1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 - P_2 & P_2 & 0 & 0 \\ P_3 & 0 & 0 & 1 - P_3 & 0 & 0 & 0 \\ 0 & P_4 & 0 & 0 & 0 & 1 - P_4 & 0 \\ 0 & 1 - p_1 & 0 & 0 & 0 & 0 & p_1 \\ c_2 & 0 & 1 - c_2 & 0 & 0 & 0 & 0 \\ p_2 & 0 & 1 - p_2 & 0 & 0 & 0 & 0 \end{pmatrix} \end{matrix}, \quad (7.5)$$

$$\begin{cases} P_1 = \int_0^\infty (1 - F_{d1}(t)) dF_{u1}(t), \\ P_2 = \int_0^\infty (1 - F_{d2}(t)) dG_{u1}(t), \\ P_3 = \int_0^\infty (1 - F_{u2}(t)) dG_{d1}(t), \\ P_4 = \int_0^\infty (1 - G_{u2}(t)) dG_{d2}(t). \end{cases} \quad (7.6)$$

The steady-state probability $\mathbf{v}$ of the embedded DTMC can be derived. Let $h_i = \int_0^\infty (1 - H_i(t)) dt$ be the mean sojourn time in state i , then we have:

$$\begin{cases} h_{(1,1)} = \int_0^\infty (1 - F_{u1}(t))(1 - F_{d1}(t)) dt, \\ h_{(0,1)} = \int_0^\infty (1 - G_{u1}(t))(1 - F_{d2}(t)) dt, \\ h_{(1,0)} = \int_0^\infty (1 - G_{d1}(t))(1 - F_{u2}(t)) dt, \\ h_{(0,0)} = \int_0^\infty (1 - G_{u2}(t))(1 - G_{d2}(t)) dt, \\ h_{(0,1')} = h_{(0,1'')} = h_{(0,0')} = 0. \end{cases} \quad (7.7)$$

The steady-state probability π_i for the SMP is given by

$$\pi_i = \frac{v_i h_i}{\sum_j v_j h_j}. \quad (7.8)$$

7.4 Performance Analysis

The primary advantage of the dynamic retraining policy lies in its ability to provide ML engineers with fine-grained control over the frequency of retraining the upstream model through the use of monitoring metrics, such as the WACR or RR. Under the dynamic retraining policy, the service availability and continuity of the MLS vary based on the threshold chosen for the monitoring metric. For instance, if the ML engineer selects the WACR as the monitoring metric and sets the threshold to 0.8, retraining is triggered only when the WACR of the retraining result exceeds 0.8. In an extreme case where the threshold is set to 1.0, no retraining attempt would be accepted, which corresponds to a $P(T | F) = 0$. This effectively makes the policy equivalent to the conservative retraining policy. Conversely, if the threshold is set to 0, which leads to $P(T | F) = 1$ and all retraining attempts would be accepted. This transitions the policy to resemble the progressive retraining policy. By adjusting the threshold of the monitoring mechanism, the engineer can control the frequency of retraining attempts. This, in turn, influences the MLS's behavior, resulting in different levels of service availability and continuity.

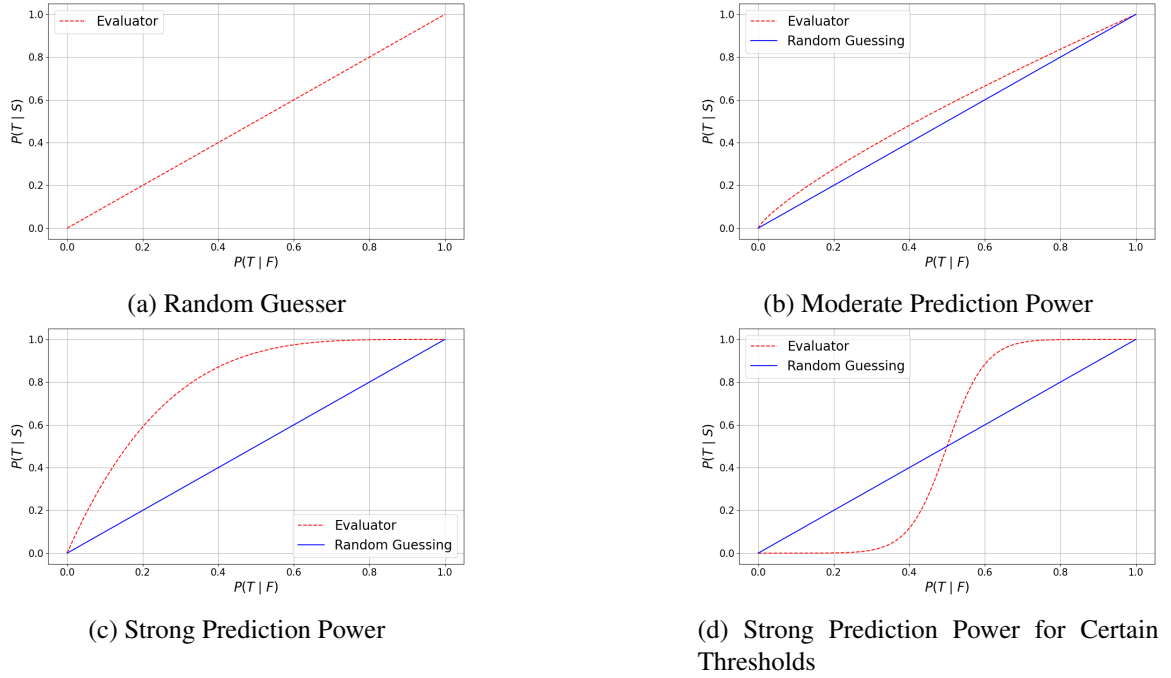


Figure 7.3: ROCs of the metric with different prediction power

The potential levels of availability and continuity achievable by the MLS depend on the validity of the monitoring metric. The validity of the metric can be evaluated using an Receiver-operating characteristic curve (ROC), which shows the inter-relationship between $P(T | S)$ and $P(T | F)$. Each point on the ROC curve corresponds to a specific threshold level on the selected monitoring metric (e.g., WACR or RR). To analyze this, we use continuous functions to simulate potential ROC curves for metrics with varying predictive power, as shown in Figure 7.3. Figure 7.3 (a) represents a

random guesser, with its ROC curve forming a diagonal line. Figure (b) shows a metric with limited predictive power, where the ROC curve lies slightly above the diagonal line. Figure (c) illustrates a metric with very strong predictive power, where the ROC curve approaches the top-left corner. Figure (d) depicts a unique ROC curve that starts flat, rises sharply in the middle, and flattens again. It suggests that the metric is particularly effective for certain thresholds within a specific range.

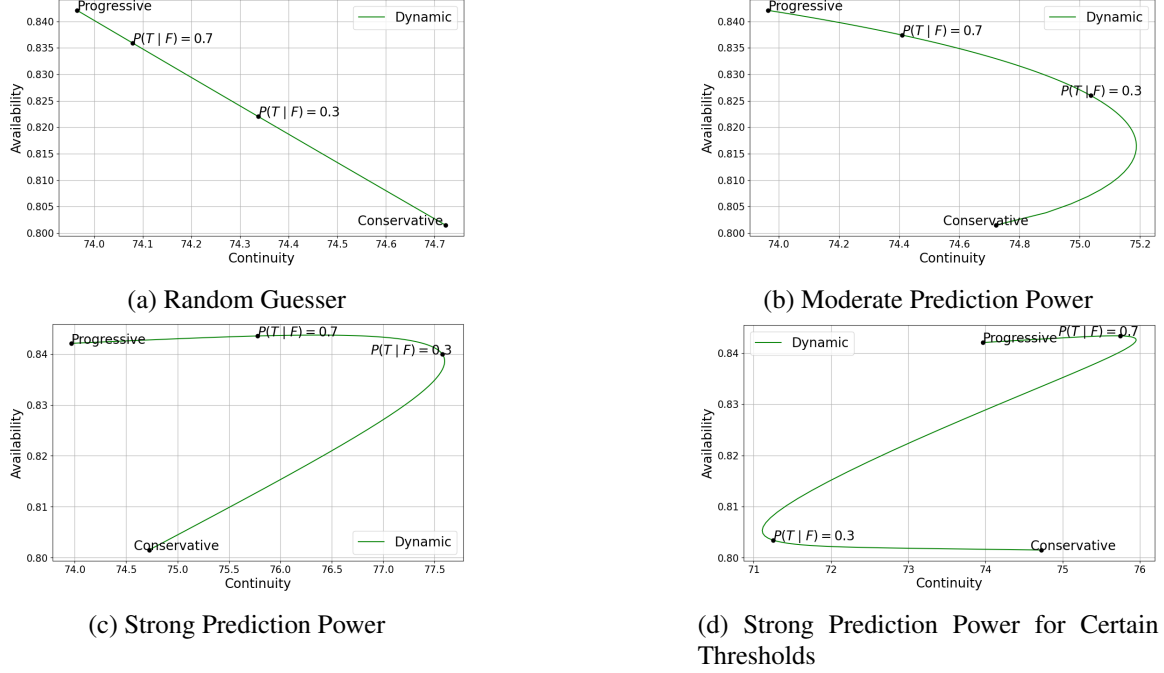


Figure 7.4: Service Availability and Service Continuity regarding to different ROCs

Figure 7.4 illustrates the availability-continuity combinations achievable by the MLS corresponding to different ROC curves using parameters shown in Table 6.2. In Figure 7.4 (a), the availability-continuity trade-off forms a nearly straight line. This indicates that even if the chosen metric lacks predictive power, the ML engineer can still balance availability and continuity by adjusting the threshold. Figure 7.4 (b) shows a curved line representing the achievable performance corresponding to a metric with moderate predictive power. Starting from the upper-left corner which represents the performance corresponding to a fully loosen threshold, by gradually tightening the threshold, continuity increases while availability decreases. However, beyond a certain point, both metrics decline. This suggests that for such a metric, the threshold should not be set too high, as stricter thresholds result in diminished performance. In Figure 7.4 (c), for a highly predictive metric, starting from the upper-left corner which represents completely loosen threshold, both availability and continuity increase initially as the threshold rises. However, beyond a critical threshold, a trade-off emerges, and eventually, both metrics decline. This indicates that overly strict thresholds should be avoided. Also, with a metric of strong prediction power, the MLS can achieve a availability-continuity combination lies in the top-right corner, compared to the other metrics. This shows that a monitoring mechanism with high prediction power can boost the performance of the MLS. In Figure

7.4 (d), adjusting the threshold either upward from a low value or downward from a high value can improve one metric without sacrificing the other. This suggests that the threshold should be set to a moderate value, neither too high nor too low, to achieve optimal results.

7.5 Utility Analysis

7.5.1 Utility Function

In system management, multiple quality attributes, such as performance, availability, and security, are in trade-off relation and need to be considered together. These quality attributes are often encoded in a utility function, i.e., a single aggregate function whose expected value should be maximized by the system [44] [45] [46]. The utility function in the context of system engineering provide the objective function for self-optimization, mapping state of the system into a scalar value [47].

In our modeling, the service availability and continuity describe the performance aspect for the ML engineer in terms of managing the system. Therefore, ML engineer can define the utility function of the MLS as $u(A, C)$, in which A represents the service availability and C represents the service continuity. Under dynamic retraining policy, by the change in the threshold for the selected monitoring metrics, the MLS experience changes in the frequency of upstream model recovery. This leads to different availability and continuity combinations under dynamic retraining policy. Among these combinations of A and C , due to the property of SMP, we have the following theorem.

Theorem 1. *Given a continuous utility function with service availability and continuity as variables, the maximum utility can always be found for the system under dynamic retraining policy.*

Proof. The accuracy of the monitoring mechanism has two dimensions, $P(T | S)$ and $P(T | F)$, in which both dimensions are bounded and closed on $[0, 1]$. Also, in the TPM P matrix, both p_1 and p_2 are continuous with regard to $P(T | S)$ and $P(T | F)$. The steady-state probability of the embedded DTMC v is solved with the linear system $v = vP$ and $ve^T = 1$. This is equivalent to solving the linear system: $v(I - P) = 0$, subject to $ve^T = 1$, where I is the identity matrix. For a Markov chain, P is a stochastic matrix (its rows sum to 1). Assuming P is irreducible and aperiodic, there is a unique steady-state vector v . The kernel of $I - P$ is generated by continuous basis, so the unique intersection of the kernel and the affine space determined by $ve^T = 1$ is also continuous to TPM's entry p_1 and p_2 . Since p_1, p_2 are continuous functions of for $P(T | S)$ and $P(T | F)$, any changes in $P(T | S)$ and $P(T | F)$ result in continuous changes to v . The normalization constraint $ve^T = 1$ is a linear operation and does not introduce discontinuities. Hence, the normalization preserves the continuity of v . Since the availability and continuity is also a continuous function of v , by the transferability of the continuity, the utility function is also continuous with regard to $P(T | S)$ and $P(T | F)$. By the extreme value theorem, because the domain of $P(T | S)$ and $P(T | F)$ are bounded and closed, it guarantees that the utility function can reach a global maximum and global minimum within the domain. $\square$

With the above theorem, dynamic retraining policy provides a key advantage: Besides enabling a range of availability and continuity combinations beyond those achievable with progressive or

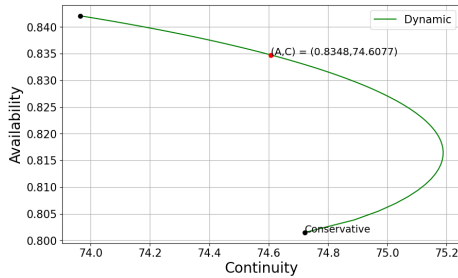
conservative retraining policies, dynamic retraining policy allows for the identification of specific thresholds that maximize the utility of the MLS.

7.5.2 Maximum Utility Analysis

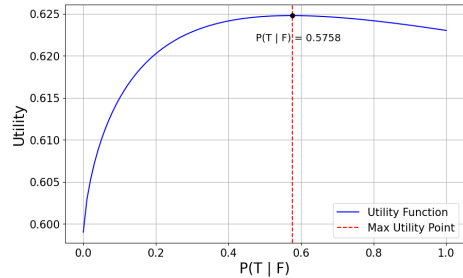
We have shown that under the dynamic retraining policy, there is always a global maximum related to the utility of the MLS. However, in different scenarios, utility functions could also be defined differently to suit the need for different MLS. In this section, to analyze the effect of different types of utility function, we first suppose that the monitoring mechanism has a moderate prediction power, and the ROC of the monitoring metric follows Fig 7.3(b). Therefore, the achievable availability-continuity combinations follow Fig 7.4(b).

We begin by analyzing the case where the utility function is convex. For a convex utility function $u(A, C)$, for all points (A_1, C_1) and (A_2, C_2) , and for any $\lambda \in [0, 1]$, we have the property such that $u(\lambda A_1 + (1 - \lambda)A_2, \lambda C_1 + (1 - \lambda)C_2) \leq \lambda u(A_1, C_1) + (1 - \lambda)u(A_2, C_2)$. Convex utility functions promote balanced trade-offs between A and C , rewarding simultaneous improvements in both metrics. Such functions reflect risk-averse behavior, where achieving moderate values for both A and C is more desirable than extreme trade-offs (e.g., very high A paired with very low C). MLSs suitable for convex utility functions prioritize both long-term performance and task-oriented stability. For instance, a recommendation system is an ideal use case. In this context, high availability ensures the system is consistently able to recommend relevant items across sessions, while high continuity ensures stable and accurate recommendations within each session. Both metrics are critical for maintaining user satisfaction and trust.

Figure 7.5 illustrates the performance of an MLS optimized for a convex utility function of the form: $u(A, C) = 0.5 \cdot A^2 + 0.5 \cdot C^2$. This utility function equally values availability and continuity, emphasizing the importance of achieving high performance in both. The maximum utility is achieved at a threshold of $P(T | F) = 0.5758$, where both availability and continuity attain relatively high values, reflecting the system's balanced performance.



(a) Availability and Continuity as threshold changes

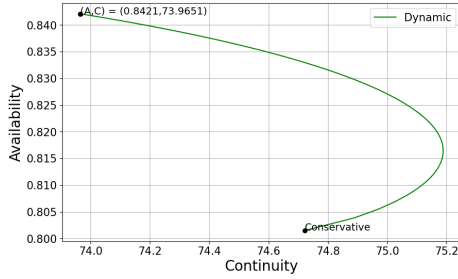


(b) Utility as threshold changes

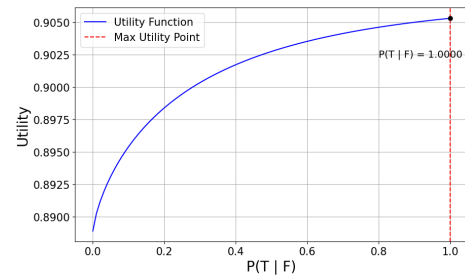
Figure 7.5: Service Availability and Service Continuity when utility function is convex

Next, we consider the case where the utility function is concave. For a concave utility function $u(A, C)$, for all points (A_1, C_1) and (A_2, C_2) , and for any $\lambda \in [0, 1]$, we have the property such that $u(\lambda A_1 + (1 - \lambda)A_2, \lambda C_1 + (1 - \lambda)C_2) \geq \lambda u(A_1, C_1) + (1 - \lambda)u(A_2, C_2)$. A concave

utility function reflects a preference for extremes (e.g., high A or high C) rather than balance. This reflects a risk-seeking behavior, where large trade-offs between A and C are desirable. One use case for such a utility function is in high-frequency trading systems, where engineers may prioritize short-term stability to achieve immediate profit, even at the expense of long-term accuracy. Figure 7.6 illustrates the performance of an MLS optimized for a concave utility function of the form: $u(A, C) = 0.8\sqrt{A} + 0.2\sqrt{C}$. In this case, the utility function assigns more importance to availability. As a result, the dynamic retraining policy aligns entirely with the progressive retraining policy, where the system selects the highest possible availability. This outcome corresponds to a completely loosened threshold, as maximizing availability dominates the trade-off.

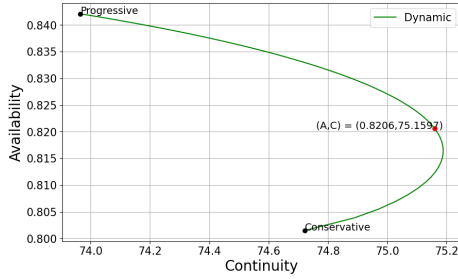


(a) Availability and Continuity as threshold changes

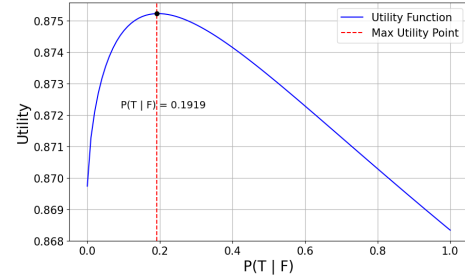


(b) Utility as threshold changes

Figure 7.6: Service Availability and Service Continuity when utility function is concave and availability is more important



(a) Availability and Continuity as threshold changes



(b) Utility as threshold changes

Figure 7.7: Service Availability and Service Continuity when utility function is concave and continuity is more important

On the other hand, Figure 7.7 shows the performance of an MLS optimized for a concave utility function of the form: $u(A, C) = 0.2\sqrt{A} + 0.8\sqrt{C}$. Here, the utility function places greater importance on continuity. The resulting threshold achieves a system with very high continuity but only moderate availability. Unlike the progressive retraining policy, this policy does not converge to the conservative retraining policy. This is because, due to the properties of the monitoring mechanism, further tightening the threshold when it is already high leads to a simultaneous decrease in both

availability and continuity. This trade-off, and the corresponding system behavior, was discussed in the previous subsection.

7.6 Strategy Analysis

Section 7.4 demonstrated that under the dynamic retraining policy, the system’s availability and continuity can vary based on the ROC characteristics of the monitoring metrics. Section 7.5 established that for a continuous utility function, it is always possible to identify a maximum utility and the corresponding thresholds. Also, depends on the different use settings, the utility function can be modified to reflect the need for the system. Together, these analyses provide a practical framework for real-world MLS implementations, enabling the selection of monitoring thresholds tailored to specific use cases. A practical strategy for the MLS engineer can be divided into the following 4 steps.

Step 1: Model the MLS The first step for the ML engineer is to gather performance data for each component of the MLS and identify the timestamps of imperfect retraining events. This data helps estimate the parameters for the state transition distributions. For demonstration purposes, we assume the engineer is managing a two-component, sequentially dependent MLS, with estimated parameters as presented in Tables 6.1 and 6.2.

Step 2: Choose the Utility Function Although we have analyzed how different utility functions could affect the ideal choice of the threshold, in practice, utility functions are often defined as the weighted sum of relevant objectives in terms of system engineering [48] [49], and the weights are often defined manually instead of automatically [46]. Therefore, here we assumed in the demonstrated MLS, this straightforward approach is adopted. A potential strategy to determine the weights of A and C would be to break down the service availability to metrics such as average satisfaction rate and successful prediction rate, and service continuity to metric such as average successful task-duration and performance interruption frequency. Stakeholders should assess the importance of these metrics relative to Key Performance Indicators (KPIs) and assign weights to availability and continuity accordingly. We define the utility function as: $u(A, C) = 0.5 \cdot A^* + 0.5 \cdot C^*$, where A^* and C^* represents the normalized form of A and C .

Step 3: Evaluate the Monitoring Mechanism Selecting an effective monitoring metric requires historical data to validate its effectiveness. Using prediction results recorded at various thresholds, the engineer can construct a ROC curve with $P(T | S)$ (true positives) and $P(T | F)$ (false positives) as the axes. Depending on the context of MLS, WACR or RR can be adopted. As we have shown, even a random monitoring mechanism enables trade-offs between metrics. In our case, the chosen mechanism demonstrates moderate predictive power, as shown in Figure 7.8. Raising the threshold reduces both $P(T | S)$ and $P(T | F)$, with the curve slightly above the diagonal line, which represents random guessing.

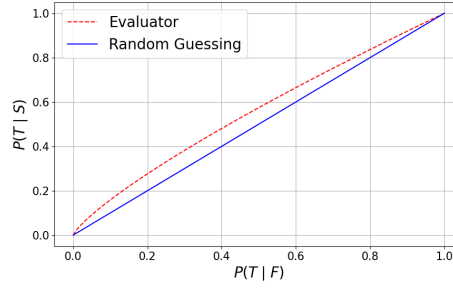
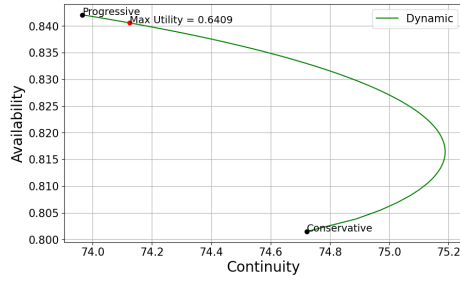
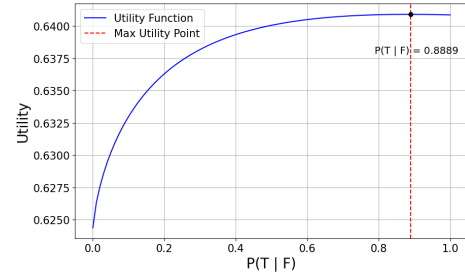


Figure 7.8: The ROC of the evaluator.

Step 4: Select the Threshold Using the ROC in Figure 7.8, the engineer can map changes in $P(T | S)$ to $P(T | F)$. These values are then used in Equations 7.3 and 7.4 to calculate availability and continuity for various thresholds. Figure 7.9(a) illustrates how threshold choices affect service availability and continuity, while Figure 7.9(b) shows utility as a function of $P(T | F)$. The utility reaches its maximum value of 0.6409 at $P(T | F) = 0.8899$. This indicates the threshold at which system performance is optimized, and the ML engineer should set this threshold for the monitoring mechanism.



(a) Availability and Continuity as threshold changes



(b) Utility as threshold changes

Figure 7.9: Service Availability and Service Continuity when evaluation method has moderate performance

Chapter 8

Conclusion

This thesis addressed the issue of imperfect retraining in MLSs, particularly the challenges posed by entangled enhancement. The problem of imperfect retraining is formally defined, supported by real-world MLS examples. Using state-space modeling, various retraining strategies are proposed and systematically compared. These analyses and comparisons provide valuable insights into mitigating the challenges of imperfect retraining, offering practical solutions for improving MLS performance in real-world applications.

In Chapter 3, we defined the problem of imperfect retraining and its implications for MLSs. In Chapter 4, through case studies involving 3D object detection and sentence classification systems, we highlighted how retraining individual components can fail to enhance, or even degrade, overall system performance. This demonstrated the need for tailored retraining policies to mitigate such risks effectively.

In chapter 5, we proposed two foundational retraining policies, the *progressive retraining policy* and the *conservative retraining policy*, to maintain MLS performance in the face of imperfect retraining. Using CTMCs, we modeled the state transitions of MLSs consists of two sequentially-dependent ML models under these policies and conducted sensitivity analyses to evaluate their impact on service availability. The results revealed distinct advantages of each policy under varying conditions, providing valuable insights for selecting appropriate maintenance strategies.

Recognizing the limitations of CTMCs in capturing non-exponential state transition times, we extended our modeling approach using SMPs in Chapter 6. This allowed us to analyze retraining dynamics more accurately and introduced the novel metric of *service continuity*, which quantifies the uninterrupted time MLSs maintain acceptable performance levels. Our findings showed a trade-off between service availability and service continuity under the two baseline policies, highlighting the need for a more flexible approach.

To address this trade-off, we proposed the *dynamic retraining policy* in Chapter 7, which integrates the strengths of both baseline policies. By dynamically adjusting retraining frequency based on system monitoring, this policy enables MLS managers to balance service availability and continuity effectively. Numerical analyses demonstrated the potential of dynamic retraining policy to maximize utility across diverse operational scenarios, offering a robust solution to the challenges posed by imperfect retraining.

For future work, several avenues remain to be explored. The thesis mainly focused on a simple

MLS comprising two dependent components, where the system's output is determined primarily by the downstream model. While this configurations can effectively capture the essential dynamics of imperfect retraining caused by component entanglement, the real-world MLSs may involve more than two components. Such configurations could introduce additional layers of complexity, including interdependencies and cascading effects, which should be addressed with a potentially larger state-space modeling. Additionally, further investigation is needed into the nuances of imperfect recovery. For example, future work could identify and model specific recovery states that is used to rollback the model version after receiving some negative feedback from the users. Compared to dynamic retraining policy which test the retraining outcome, such recovery can be seen as a more passive mechanism to mitigate the impact of imperfect retraining. Another promising direction is to examine different forms of imperfect retraining beyond the entanglement of components, potentially exploring how external factors, such as fluctuating data quality or environmental changes, influence retraining outcomes.

By addressing the challenges of imperfect retraining and proposing innovative maintenance strategies, this thesis contributes to the reliable operation of MLSs in dynamic, real-world environments. These contributions will inspire further research into maintaining and improving the performance of MLSs in the face of evolving challenges.

Acknowledgment

I would like to express my deepest gratitude to my supervisor, Professor Fumio Machida, for his guidance, encouragement, and support throughout the course of this research. His expertise and insightful advice have been instrumental in shaping my work, and I am truly grateful for the time and effort he dedicated to helping me achieve this thesis.

I am also sincerely thankful to the members of our laboratory for their constructive feedback, thoughtful critiques, and suggestions. Their collaborative spirit and intellectual contributions have significantly enriched my work, enabling me to refine my ideas and improve the quality of this thesis.

Finally, I wish to acknowledge the dedicated staff at Tsukuba University. Their professionalism, kindness, and assistance have greatly facilitated my academic experience, and I am deeply appreciative of their efforts.

References

- [1] Michael I Jordan and Tom M Mitchell. Machine learning: Trends, perspectives, and prospects. *Science*, Vol. 349, No. 6245, pp. 255–260, 2015.
- [2] Fuxun Yu, Di Wang, Longfei Shangguan, Minjia Zhang, Xulong Tang, Chenchen Liu, and Xiang Chen. A survey of large-scale deep learning serving system optimization: Challenges and opportunities. *arXiv preprint arXiv:2111.14247*, 2021.
- [3] Hao Fang, Saurabh Gupta, Forrest Iandola, Rupesh K Srivastava, Li Deng, Piotr Dollár, Jianfeng Gao, Xiaodong He, Margaret Mitchell, John C Platt, et al. From captions to visual concepts and back. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1473–1482, 2015.
- [4] Léon Bottou, Jonas Peters, Joaquin Quiñero-Candela, Denis X Charles, D Max Chickering, Elon Portugaly, Dipankar Ray, Patrice Simard, and Ed Snelson. Counterfactual reasoning and learning systems: The example of computational advertising. *Journal of Machine Learning Research*, Vol. 14, No. 11, 2013.
- [5] Jiri Hron, Karl Krauth, Michael Jordan, and Niki Kilbertus. On component interactions in two-stage recommender systems. *Advances in neural information processing systems*, Vol. 34, pp. 2744–2757, 2021.
- [6] Jose G Moreno-Torres, Troy Raeder, Rocío Alaiz-Rodríguez, Nitesh V Chawla, and Francisco Herrera. A unifying view on dataset shift in classification. *Pattern recognition*, Vol. 45, No. 1, pp. 521–530, 2012.
- [7] Yingzhe Lyu, Heng Li, Mohammed Sayagh, Zhen Ming Jiang, and Ahmed E Hassan. An empirical study of the impact of data splitting decisions on the performance of aiops solutions. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, Vol. 30, No. 4, pp. 1–38, 2021.
- [8] Zhengji Wang and Fumio Machida. Maintaining performance of a machine learning system against imperfect retraining. In *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*, pp. 1782–1787. IEEE, 2024.
- [9] Besmira Nushi, Ece Kamar, Eric Horvitz, and Donald Kossmann. On human intellect and machine failures: Troubleshooting integrative machine learning systems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 31, 2017.

- [10] Gavin C Cawley and Nicola LC Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *The Journal of Machine Learning Research*, Vol. 11, pp. 2079–2107, 2010.
- [11] David Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. *Advances in neural information processing systems*, Vol. 28, , 2015.
- [12] Ruihan Wu, Chuan Guo, Awni Hannun, and Laurens van der Maaten. Fixes that fail: Self-defeating improvements in machine-learning systems. *Advances in Neural Information Processing Systems*, Vol. 34, pp. 11745–11756, 2021.
- [13] Haixun Wang, Wei Fan, Philip S Yu, and Jiawei Han. Mining concept-drifting data streams using ensemble classifiers. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*, pp. 226–235, 2003.
- [14] Haixun Wang, Philip S Yu, and Jiawei Han. Mining concept-drifting data streams. *Data Mining and Knowledge Discovery Handbook*, pp. 789–802, 2010.
- [15] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pp. 291–300. IEEE, 2019.
- [16] Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. Data management challenges in production machine learning. In *Proceedings of the 2017 ACM International Conference on Management of Data*, pp. 1723–1726, 2017.
- [17] Rafal Kocielnik, Saleema Amershi, and Paul N Bennett. Will you accept an imperfect ai? exploring designs for adjusting end-user expectations of ai systems. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pp. 1–14, 2019.
- [18] Nipuni Hewage and Dulani Meedeniya. Machine learning operations: A survey on mlops tool support. *arXiv preprint arXiv:2202.10169*, 2022.
- [19] Bojan Karlaš, Matteo Interlandi, Cedric Renggli, Wentao Wu, Ce Zhang, Deepak Mukunthu Iyappan Babu, Jordan Edwards, Chris Lauren, Andy Xu, and Markus Weimer. Building continuous integration services for machine learning. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2407–2415, 2020.
- [20] Dominik Kreuzberger, Niklas Kühl, and Sebastian Hirschl. Machine learning operations (mlops): Overview, definition, and architecture. *IEEE access*, Vol. 11, pp. 31866–31879, 2023.

- [21] Alexander D’Amour, Katherine Heller, Dan Moldovan, Ben Adlam, Babak Alipanahi, Alex Beutel, Christina Chen, Jonathan Deaton, Jacob Eisenstein, Matthew D Hoffman, et al. Underspecification presents challenges for credibility in modern machine learning. *Journal of Machine Learning Research*, Vol. 23, No. 226, pp. 1–61, 2022.
- [22] Fumio Machida, Jianwen Xiang, Kumiko Tadano, and Yoshiharu Maeno. Lifetime extension of software execution subject to aging. *IEEE Transactions on Reliability*, Vol. 66, No. 1, pp. 123–134, 2016.
- [23] Ambuj Goyal, Stephen S Lavenberg, and Kishor Shridharbhai Trivedi. Probabilistic modeling of computer system availability. *Annals of Operations Research*, Vol. 8, pp. 285–306, 1987.
- [24] YUN LIU KISHOR S TRIVEDI. Survivability quantification: The analytical modeling approach. *International Journal of Performability Engineering*, Vol. 2, No. 1, p. 29, 2006.
- [25] Liang Yin, Ricardo M Fricks, and Kishor S Trivedi. Application of semi-markov process and ctmc to evaluation of ups system availability. In *Annual Reliability and Maintainability Symposium. 2002 Proceedings (Cat. No. 02CH37318)*, pp. 584–591. IEEE, 2002.
- [26] Kalyanaraman Vaidyanathan and Kishor S Trivedi. A comprehensive model for software rejuvenation. *IEEE Transactions on Dependable and Secure Computing*, Vol. 2, No. 2, pp. 124–137, 2005.
- [27] Girish Kumar, Vipul Jain, and OP Gandhi. Availability analysis of repairable mechanical systems using analytical semi-markov approach. *Quality Engineering*, Vol. 25, No. 2, pp. 97–107, 2013.
- [28] Yan Wang, Wei-Lun Chao, Divyansh Garg, Bharath Hariharan, Mark Campbell, and Kilian Q Weinberger. Pseudo-lidar from visual depth estimation: Bridging the gap in 3d object detection for autonomous driving. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 8445–8453, 2019.
- [29] Jia-Ren Chang and Yong-Sheng Chen. Pyramid stereo matching network. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 5410–5418, 2018.
- [30] Shaoshuai Shi, Xiaogang Wang, and Hongsheng Li. Pointnet: 3d object proposal generation and detection from point cloud. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 770–779, 2019.
- [31] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *2012 IEEE conference on computer vision and pattern recognition*, pp. 3354–3361. IEEE, 2012.
- [32] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The pascal visual object classes (voc) challenge. *International journal of computer vision*, Vol. 88, pp. 303–338, 2010.

- [33] Andrea Simonelli, Samuel Rota Buló, Lorenzo Porzi, Manuel López-Antequera, and Peter Kotschieder. Disentangling monocular 3d object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1991–1999, 2019.
- [34] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of naacL-HLT*, Vol. 1, p. 2. Minneapolis, Minnesota, 2019.
- [35] Yoon Kim. Convolutional neural networks for sentence classification. In Alessandro Moschitti, Bo Pang, and Walter Daelemans, editors, *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1746–1751, Doha, Qatar, October 2014. Association for Computational Linguistics.
- [36] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pp. 785–794, 2016.
- [37] Wenting Li, Shangbing Gao, Hong Zhou, Zihe Huang, Kewen Zhang, and Wei Li. The automatic text classification method based on bert and feature union. In *2019 IEEE 25th International Conference on Parallel and Distributed Systems (ICPADS)*, pp. 774–777. IEEE, 2019.
- [38] Ahmed Cherif Mazari, Nesrine Boudoukhani, and Abdelhamid Djeflal. Bert-based ensemble learning for multi-aspect hate speech detection. *Cluster Computing*, Vol. 27, No. 1, pp. 325–339, 2024.
- [39] Szu-Yin Lin, Yun-Ching Kung, and Fang-Yie Leu. Predictive intelligence in harmful news identification by bert-based ensemble learning model with text sentiment analysis. *Information Processing & Management*, Vol. 59, No. 2, p. 102872, 2022.
- [40] David J Hurst. Operational availability modeling for risk and impact analysis. In *Annual Reliability and Maintainability Symposium 1995 Proceedings*, pp. 391–396. IEEE, 1995.
- [41] Julio Hurtado, Dario Salvati, Rudy Semola, Mattia Bosio, and Vincenzo Lomonaco. Continual learning for predictive maintenance: Overview and challenges. *Intelligent Systems with Applications*, Vol. 19, p. 200251, 2023.
- [42] Kerstin N Vokinger, Stefan Feuerriegel, and Aaron S Kesselheim. Continual learning in medical devices: Fda’s action plan and beyond. *The Lancet Digital Health*, Vol. 3, No. 6, pp. e337–e338, 2021.
- [43] Cecilia S Lee and Aaron Y Lee. Clinical applications of continual learning machine learning. *The Lancet Digital Health*, Vol. 2, No. 6, pp. e279–e281, 2020.
- [44] Funmilade Faniyi, Peter R Lewis, Rami Bahsoon, and Xin Yao. Architecting self-aware software systems. In *2014 IEEE/IFIP Conference on Software Architecture*, pp. 91–94. IEEE, 2014.

- [45] William Heaven, Daniel Sykes, Jeff Magee, and Jeff Kramer. A case study in goal-driven architectural adaptation. *Software engineering for self-adaptive systems*, pp. 109–127, 2009.
- [46] Paola Inverardi and Marco Mori. A software lifecycle process to support consistent evolutions. In *Software Engineering for Self-Adaptive Systems II: International Seminar, Dagstuhl Castle, Germany, October 24-29, 2010 Revised Selected and Invited Papers*, pp. 239–264. Springer, 2013.
- [47] William E Walsh, Gerald Tesauro, Jeffrey O Kephart, and Rajarshi Das. Utility functions in autonomic systems. In *International Conference on Autonomic Computing, 2004. Proceedings.*, pp. 70–77. IEEE, 2004.
- [48] Shang-Wen Cheng, David Garlan, and Bradley Schmerl. Architecture-based self-adaptation in the presence of multiple objectives. In *Proceedings of the 2006 international workshop on Self-adaptation and self-managing systems*, pp. 2–8, 2006.
- [49] Carlo Ghezzi and Amir Molzam Sharifloo. Dealing with non-functional requirements for adaptive systems via dynamic software product-lines. In *Software Engineering for Self-Adaptive Systems II: International Seminar, Dagstuhl Castle, Germany, October 24-29, 2010 Revised Selected and Invited Papers*, pp. 191–213. Springer, 2013.